

Since past few years, there is new way of developing software applications using **MicroServices Architecture**.

So, **what is that OLD software architecture** we used before **MicroServices**?

And, **what are the disadvantages of it** that lead to shift our focus to **MicroServices**?

Objectives

1. What is a monolithic?
2. What is a monolithic architecture?
3. Advantages & Disadvantages
4. The major challenge

monolithic

mɒnəˈlɪθɪk/Submit

adjective

1. **formed of a single large block** of stone.

"later Byzantine columns were monolithic and usually made of marble"

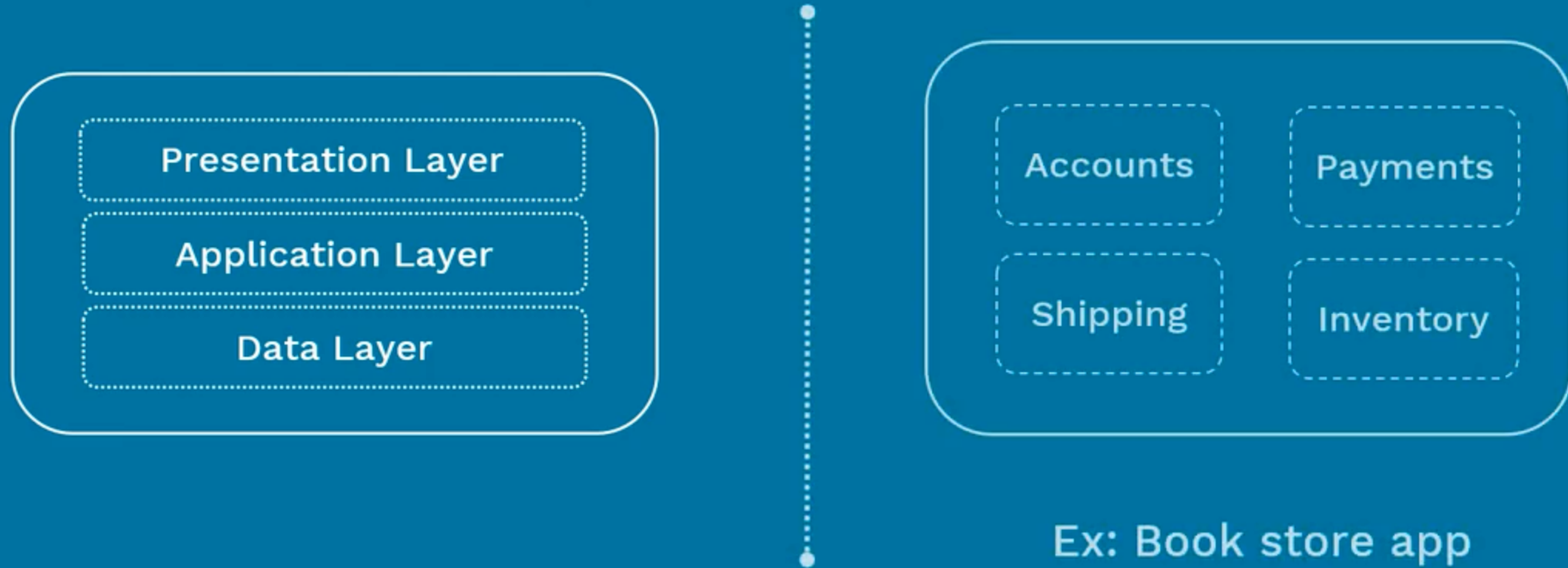
2. (of an organization or system) **large, powerful, indivisible, and slow to change.**

"rejecting any move towards a monolithic European superstate"

synonyms: **inflexible**, rigid, unbending, unchanging, intractable, **immovable**, impenetrable, fossilized, hidebound; More

Monolithic Architecture - Overview

- Self contained - packaged & deployed as single unit
- Three tiered application architecture



Advantages

- Single unit of deployment
- IDE Support

Disadvantages

- Often very large
- Technology dependency on initial decision
- Implemented using single development stack
- Frequent deployments are not practical.
- Need to scale entire application stack

The Big Problem

Internet

Scalability & Flexibility ↓

App should be **easy to deploy**, then **scale up and scale down** depending up on the traffic, and **free from all technological dependencies**.

What is that one architecture that will satisfy these business requirements?

MicroService

Creating smaller and multiple component services which are built to handled a very specific concern.

Multiple services makeup application

Characteristics

Autonomous



Specialized

Accounts

Payments

Shipping

Inventory

Advantages

Flexible Scaling

Easy Deployment

Technological Freedom

Reusable Code

Resilience

Disadvantages

Spawning

Maintenance

Duplicate Code

Network Performance

Monolithic

- Processes are tightly coupled and run as a single service
- Adding features becomes more complex
- Adds risk for application availability

MicroServices

- One service per process
- Each service performs one single function
- Can be updated, deployed, and scaled rapidly to meet demand

Where do you run MicroServices?

Physical Servers | Virtual Machines | Containers

Objectives

1. Containers in general
2. Containers in IT
3. Container Architecture
4. Advantages
5. Container Engines

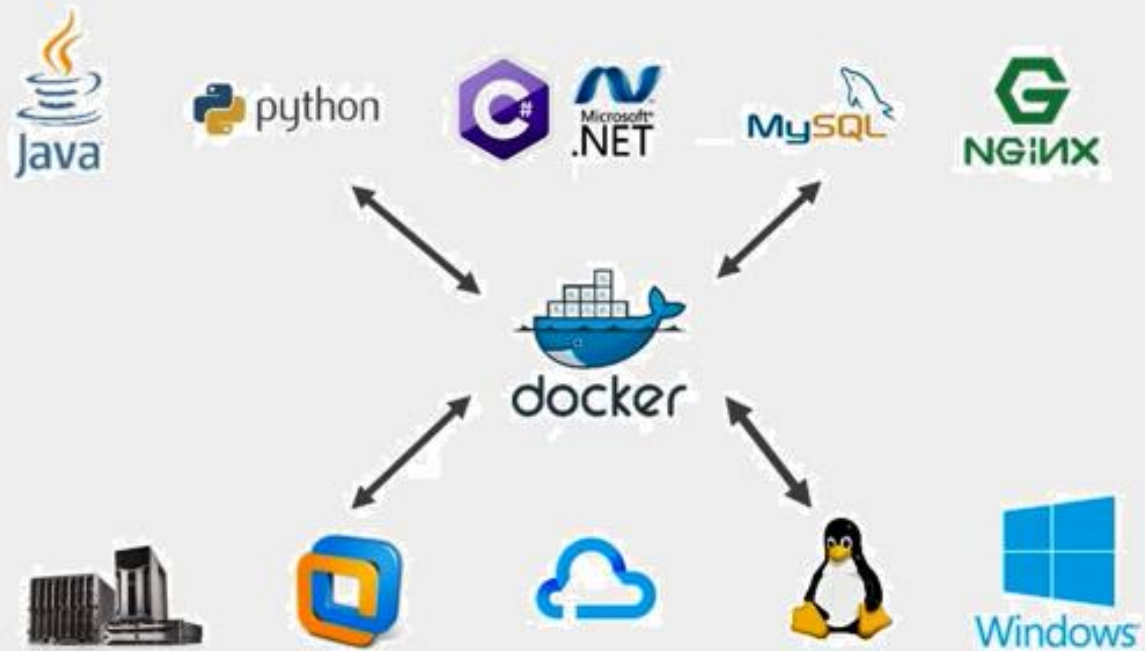




What are Containers?



What are Containers?

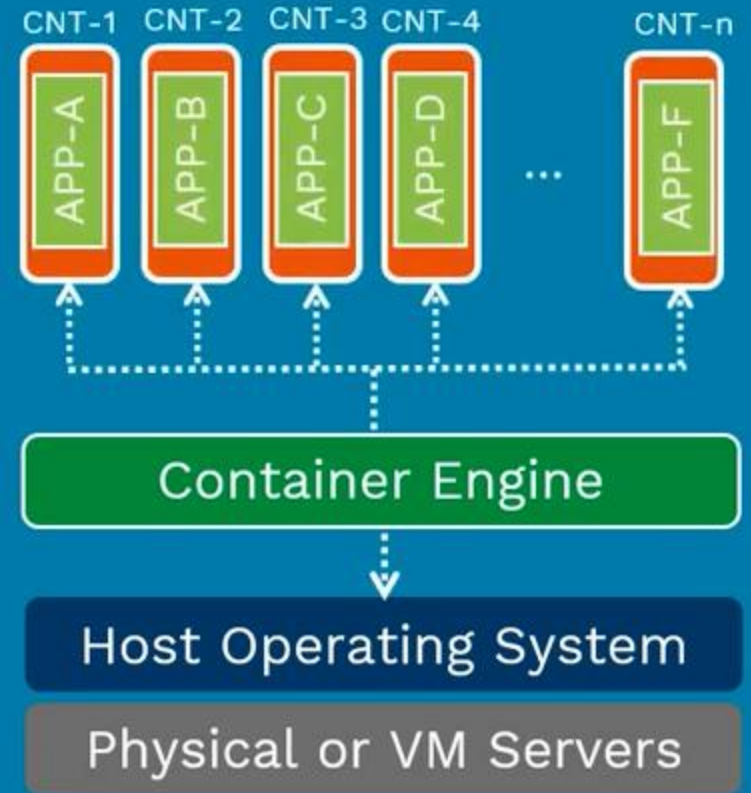


Container Architecture

- Containers virtualize at the OS level

Benefits:

- Very light weight
- Boots up in seconds
- Takes fraction of disk and memory space



Advantages

Portability

Resources efficiency

Isolation

Speed

Scaling

Container Engines

Docker

Rocket (rkt)

Objectives

1. Physical Servers
2. Virtual Machines
3. Containers
4. Virtual Machines vs. Containers

Physical Server

- Early to Mid 2000s
- One Software App per Physical Server

Drawbacks:

- CAPEX Costs
- OPEX Costs
 - Power and Cooling
 - Administration Costs
- Overpowered and massively over priced



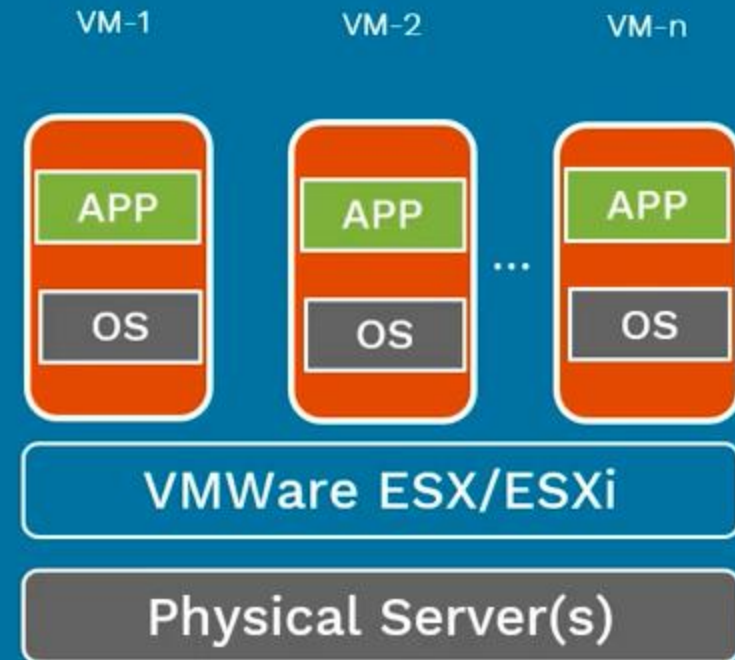
Virtual Machines

Benefits:

- Multiple VMs on Single Machine
- Consolidate apps into single physical machine
 - Cost savings
 - Faster server provisioning

Drawbacks:

- Requires compute and storage
- OS Licenses
- CAPEX costs
- OPEX costs

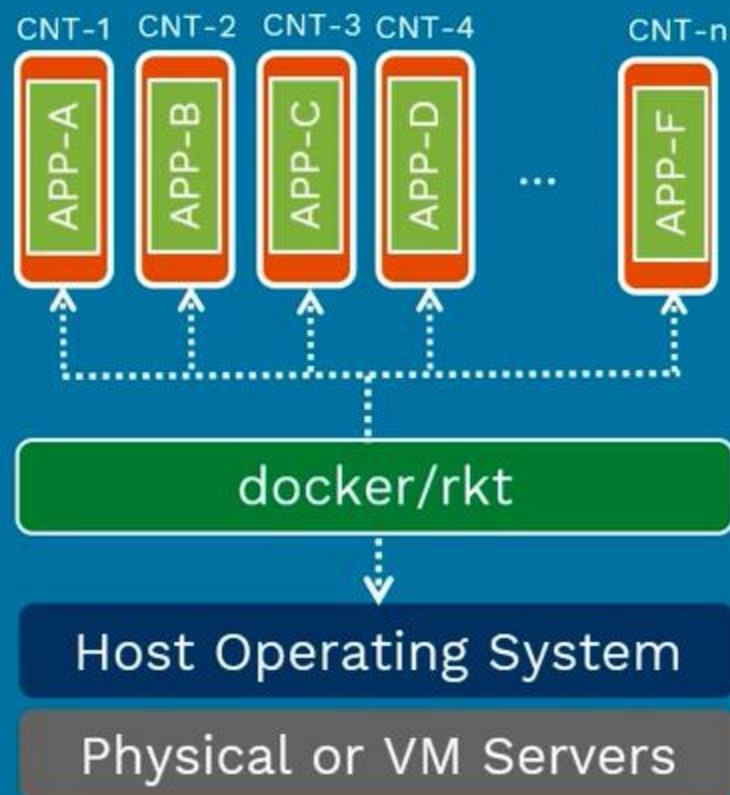


Containers

- Containers virtualize at the OS level
- Multiple containers will run on single OS

Benefits:

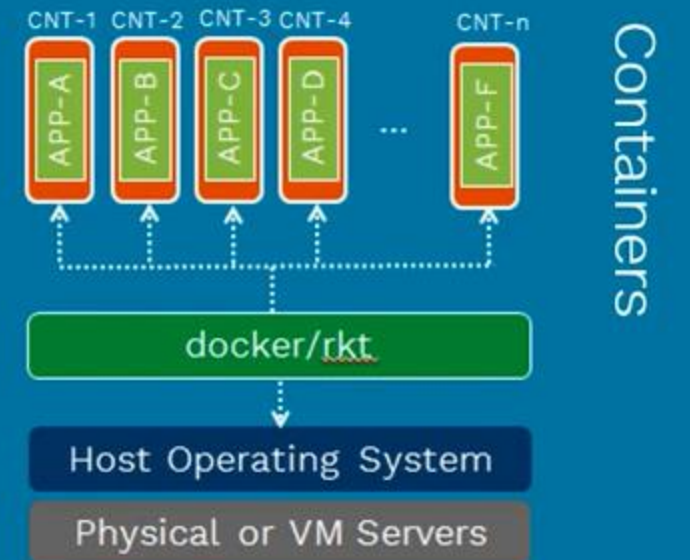
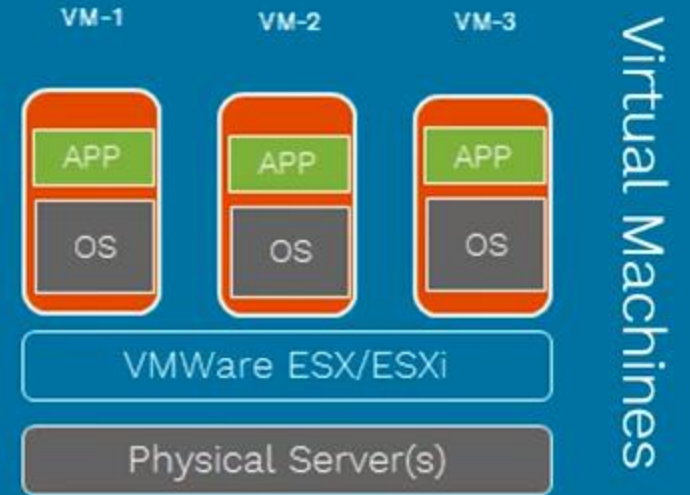
- Containers become very light weight
- It boots up in matter of seconds
- Takes fraction of disk and memory space



Virtual Machines vs. Containers

- Containers don't replace VMs
- VM virtualizes hardware
- Containers virtualizes OS
- Containers can run within VMs

Value	Containers	VMs
Boot Speed	✓	
Footprint Size	✓	
Maturity		✓
Security		✓
Ease of patching	✓	
Developer Agility	✓	



Objectives

1. What is Docker?
2. What it does?
3. Docker in Infrastructure stack
4. Docker Hub
5. Docker Project
6. Big Problem

Docker Inc.

Docker Inc.



dotCloud → Docker

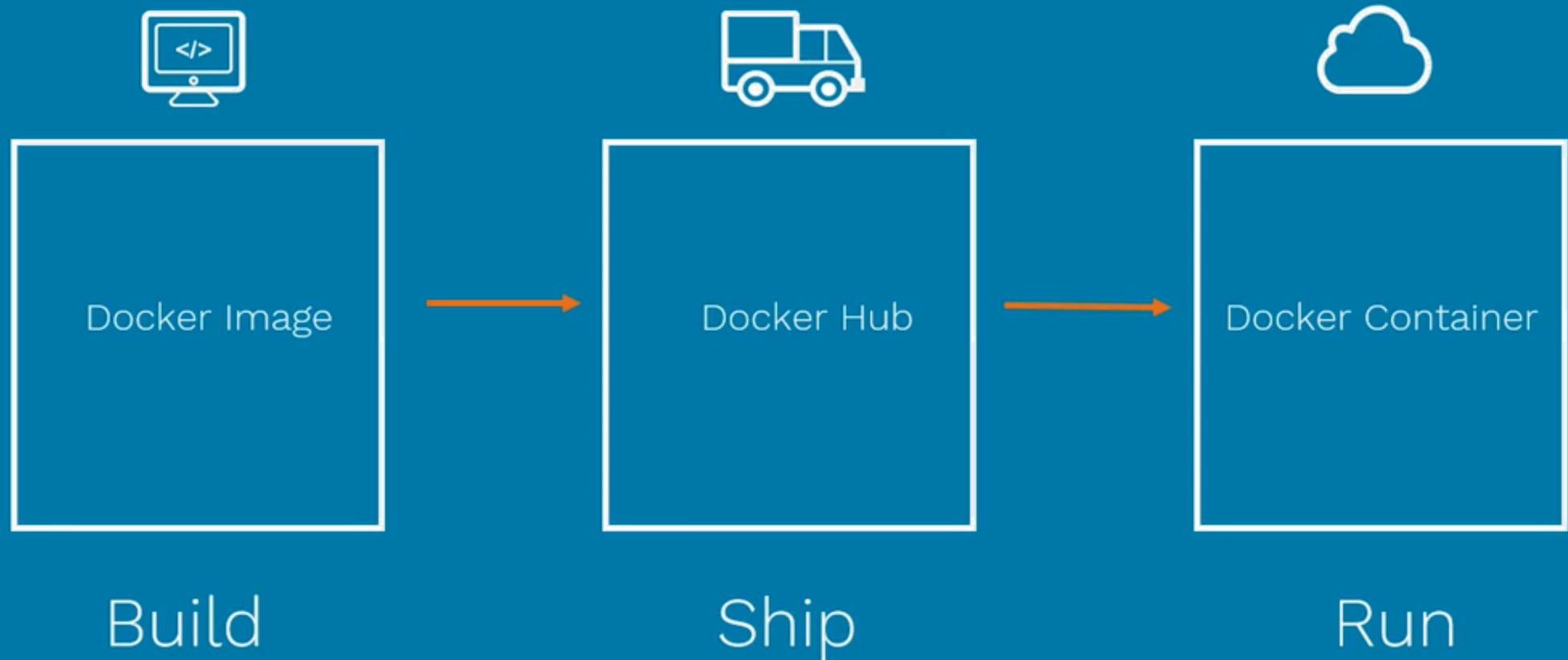
Dock ~~worker~~

\$1.5B

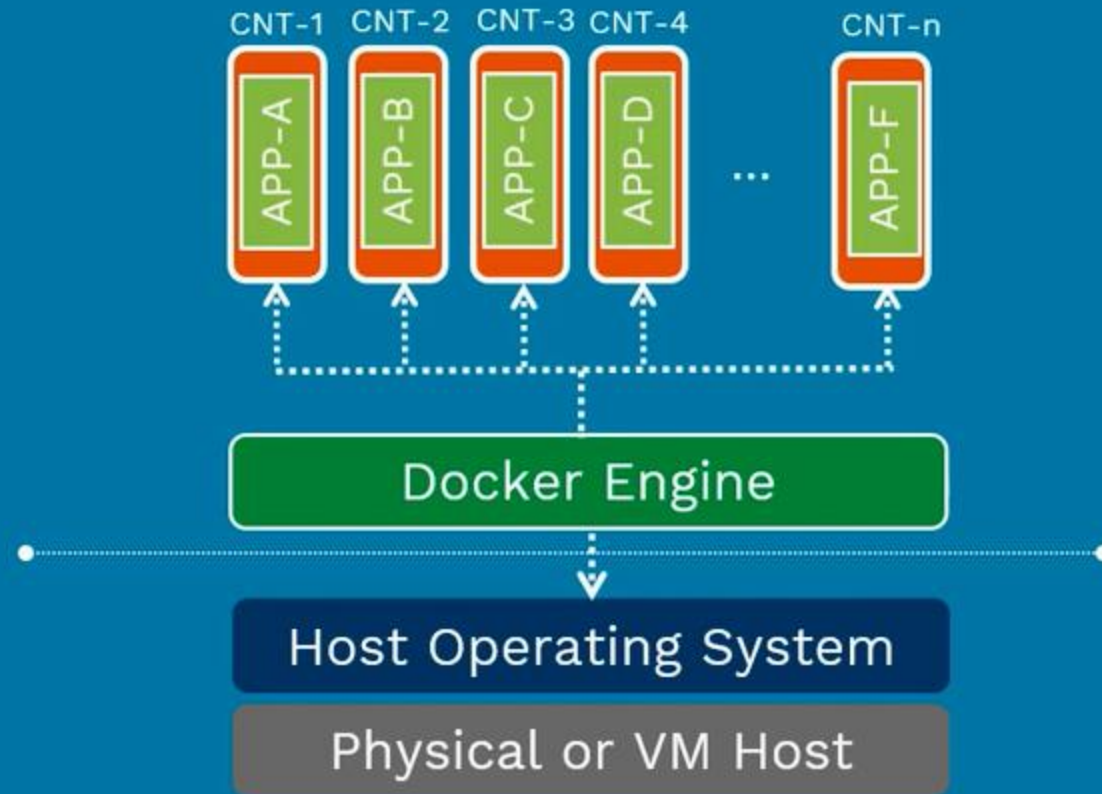


Go

What it does?



Infrastructure Stack



Docker Hub

<https://hub.docker.com/explore/>

Explore Official Repositories

 nginx official	9.3K STARS	10M+ PULLS	> DETAILS
 alpine official	4.1K STARS	10M+ PULLS	> DETAILS
 busybox official	1.3K STARS	10M+ PULLS	> DETAILS
 httpd official	1.9K STARS	10M+ PULLS	> DETAILS

The Docker Project



Search or jump to...



Pull requests

Issues

Marketplace

Explore



 [docker](#) / [docker.github.io](#)

 Watch ▾

150

 Star

1,199

 Fork

 Code

 Issues 450

 Pull requests 106

 Projects 0

 Insights

Source repo for Docker's Documentation <https://docs.docker.com>

 48,913 commits

 27 branches

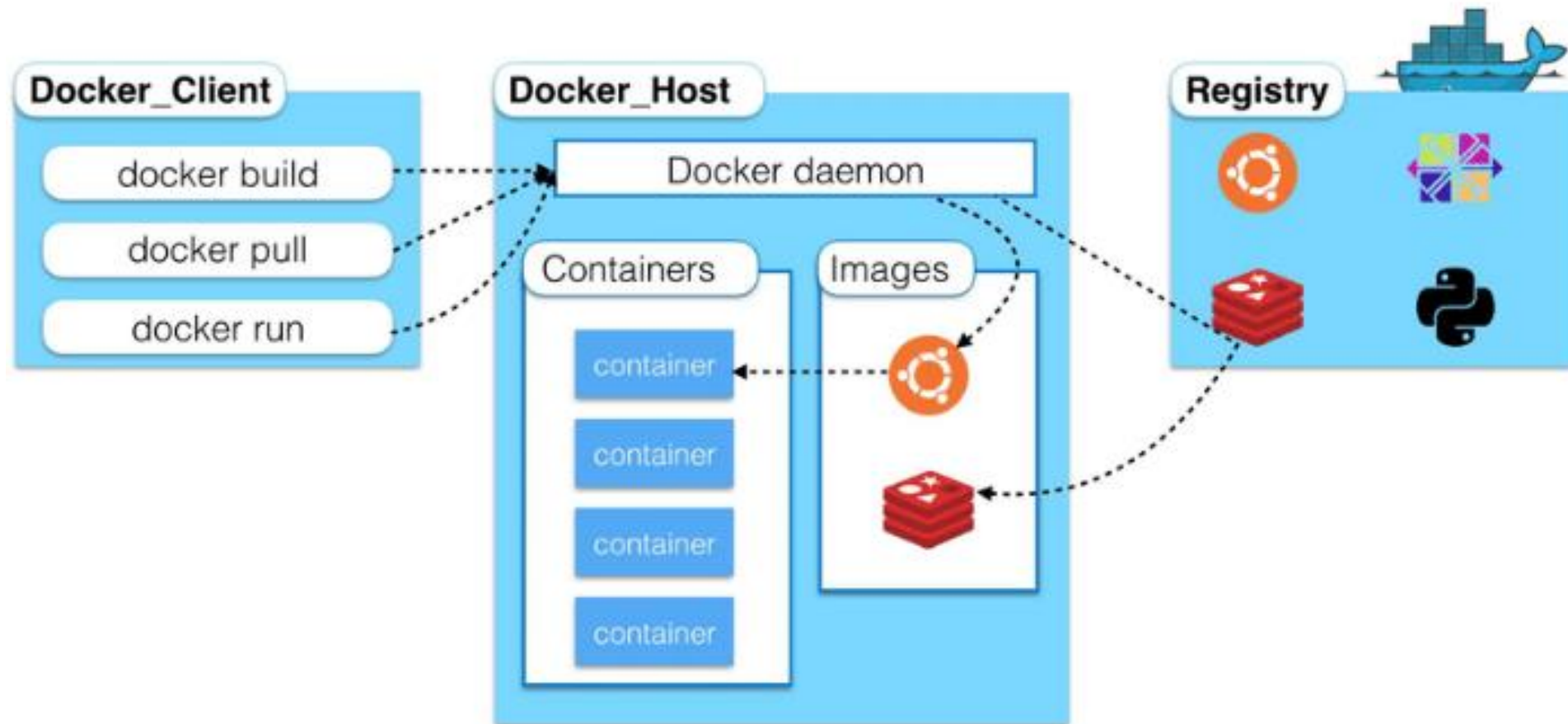
 8 releases

 2,845 contributors

 Apache-2.0

Docker Management

Docker Architecture



Basic Command for Docker

Google runs average of 2 Billion Containers every week

- How are these containers created and managed at such large scale?
- How do all these containers connect and communicate?
- How do you scale these containers as per the traffic demands?

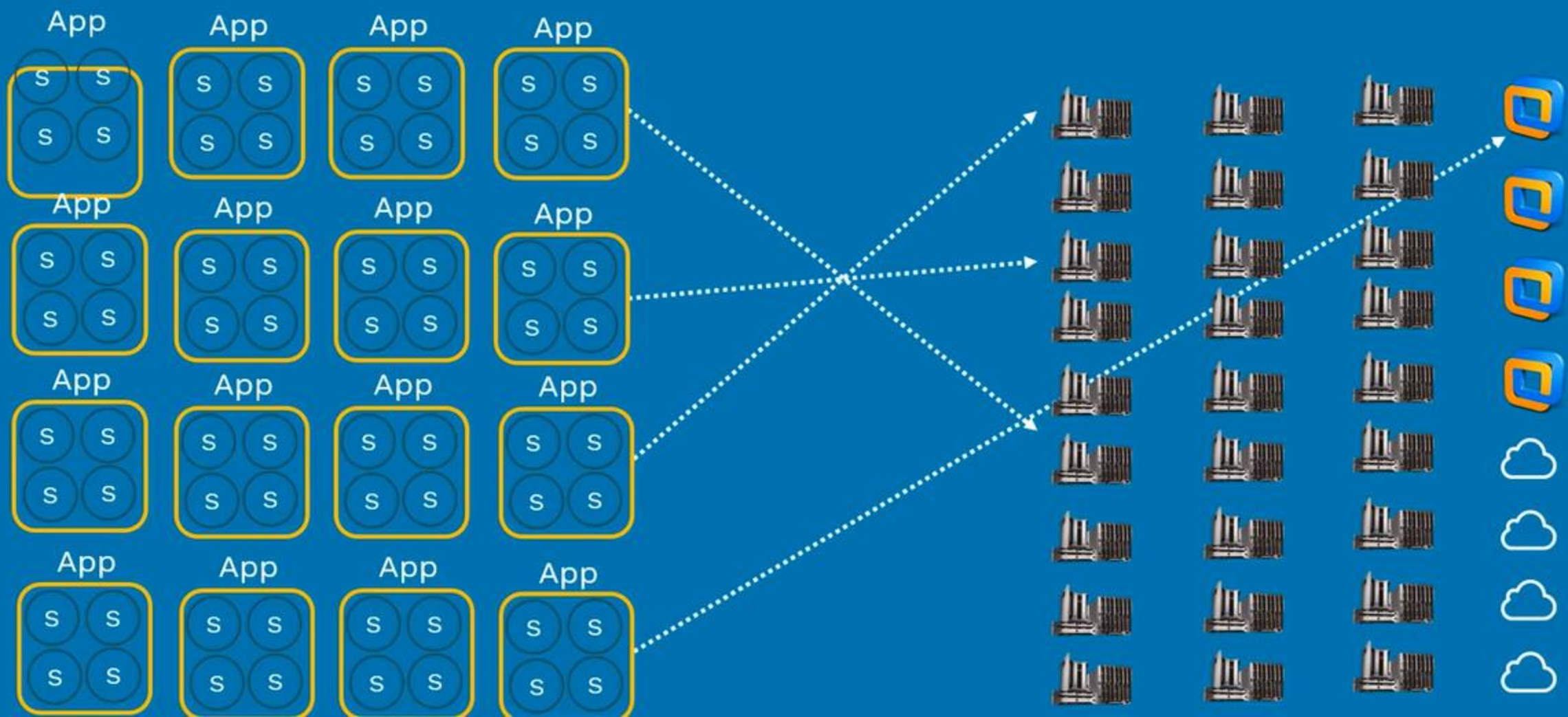
Objectives

1. When COE is not always required?
2. But, when that will be a problem?
3. What is Container Orchestration Engine?
4. COE Features

COE is not always required



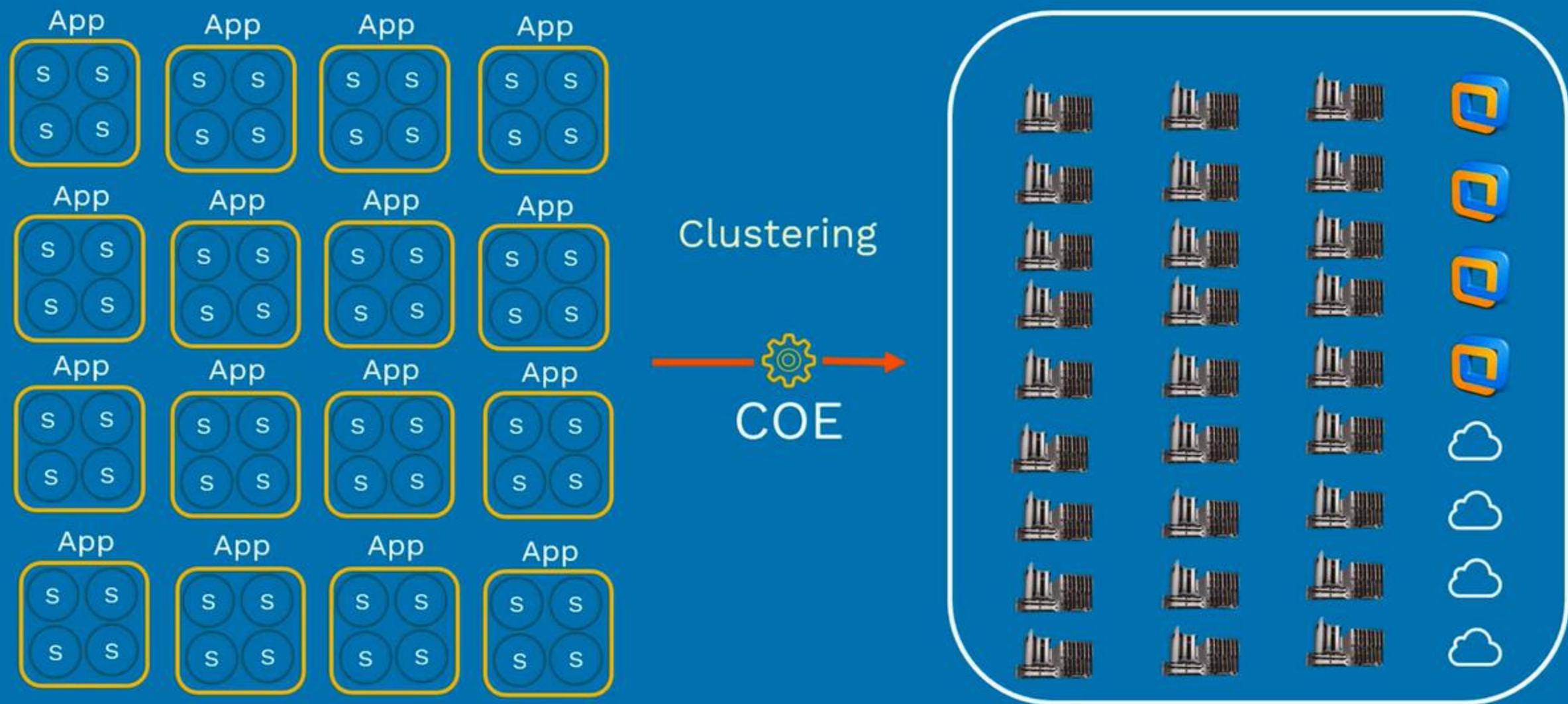
Here is the problem...



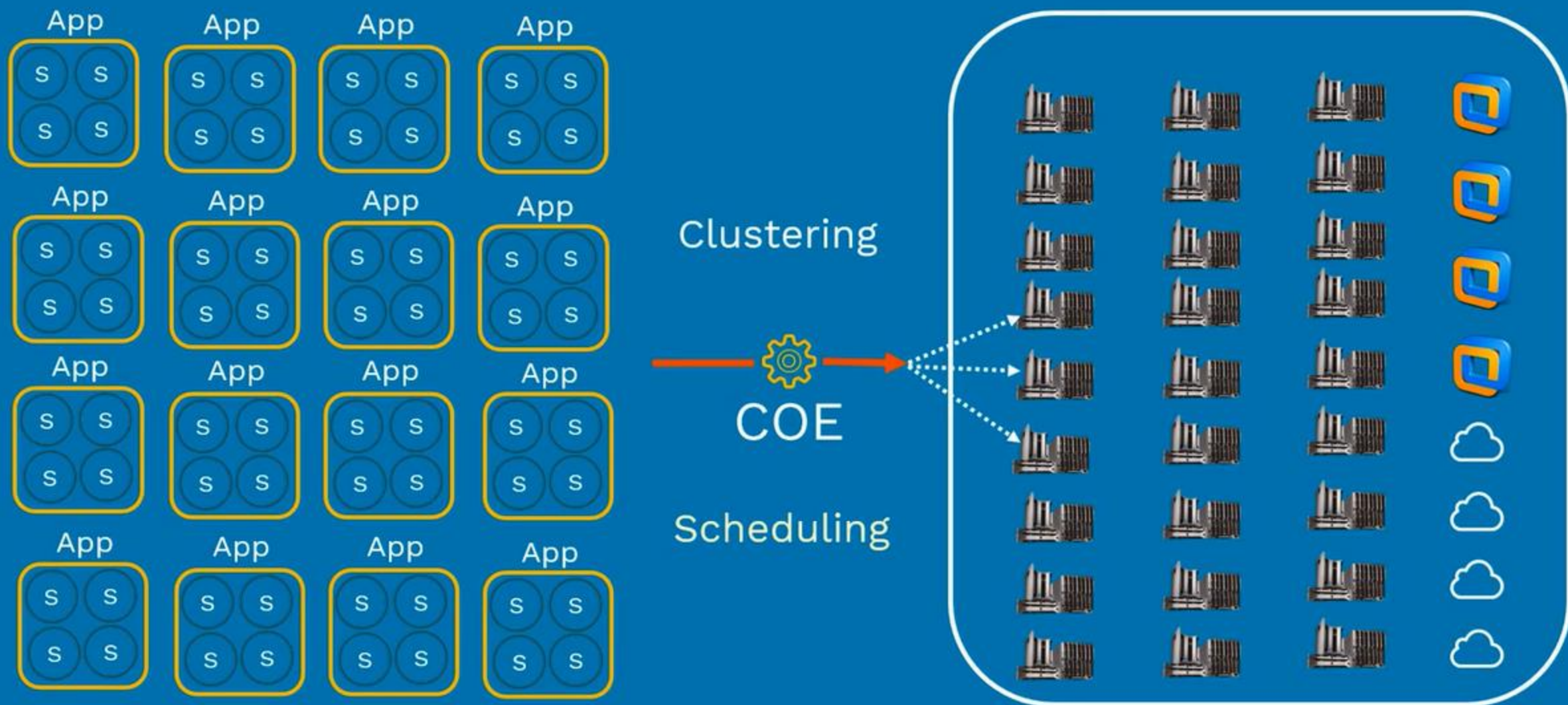
Container Orchestration Engine

Container Orchestration Engine automates **deploying, scaling, and managing** containerized applications on a group of servers.

Container Orchestration Engine

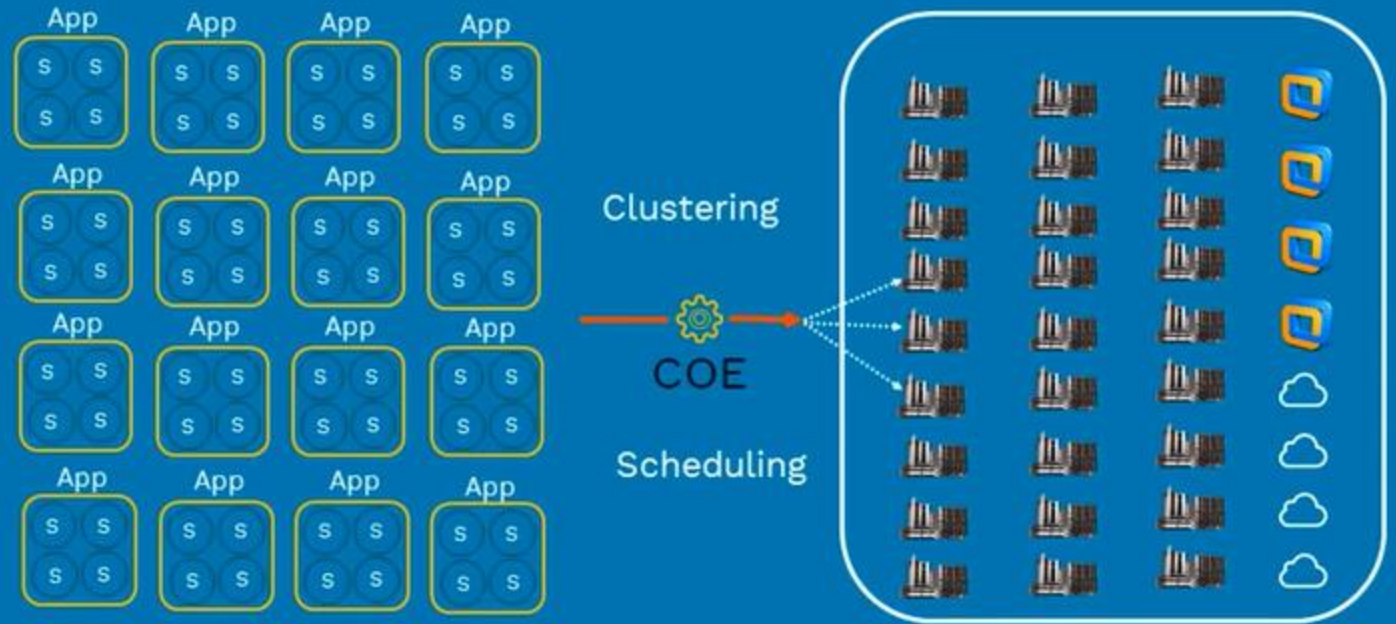


Container Orchestration Engine



COE - Features

- Clustering
- Scheduling
- Scalability
- Load balancing
- Fault tolerance
- Deployment



Objectives

1. Top 3 Container Orchestration Engines (COEs)
2. Marathon by Apache Mesos
3. Docker Swarm
4. Kubernetes
5. Cloud specific COEs
6. Choosing COEs

TOP-3 Container Orchestration Engines

Marathon

Apache Mesos



Docker Swarm

Docker Inc.



Kubernetes

CNCF



Marathon

Apache Mesos



- Apache Mesos is an open-source Cluster Manager
- Marathon framework for Containers Orchestration
- Has more than 20 frameworks
- More complex architecture than Swarm

Typical users of Apache Marathon:

- Large enterprises
- Which requires lot of compute such as Big data
- Driven by developers
- **Need an ops team to manage**

Docker Swarm

Docker Inc.



- Written in Golang
- Easy to understand and get started
- Simple architecture than Kubernetes and Mesos

Typical users of Apache Marathon are:

- Smaller teams (startup and medium-size companies)
- Typically greenfield projects
- Developer-driven teams who need to deploy products

Cloud Specific Container Services



Google
Kubernetes Engine (GKE)

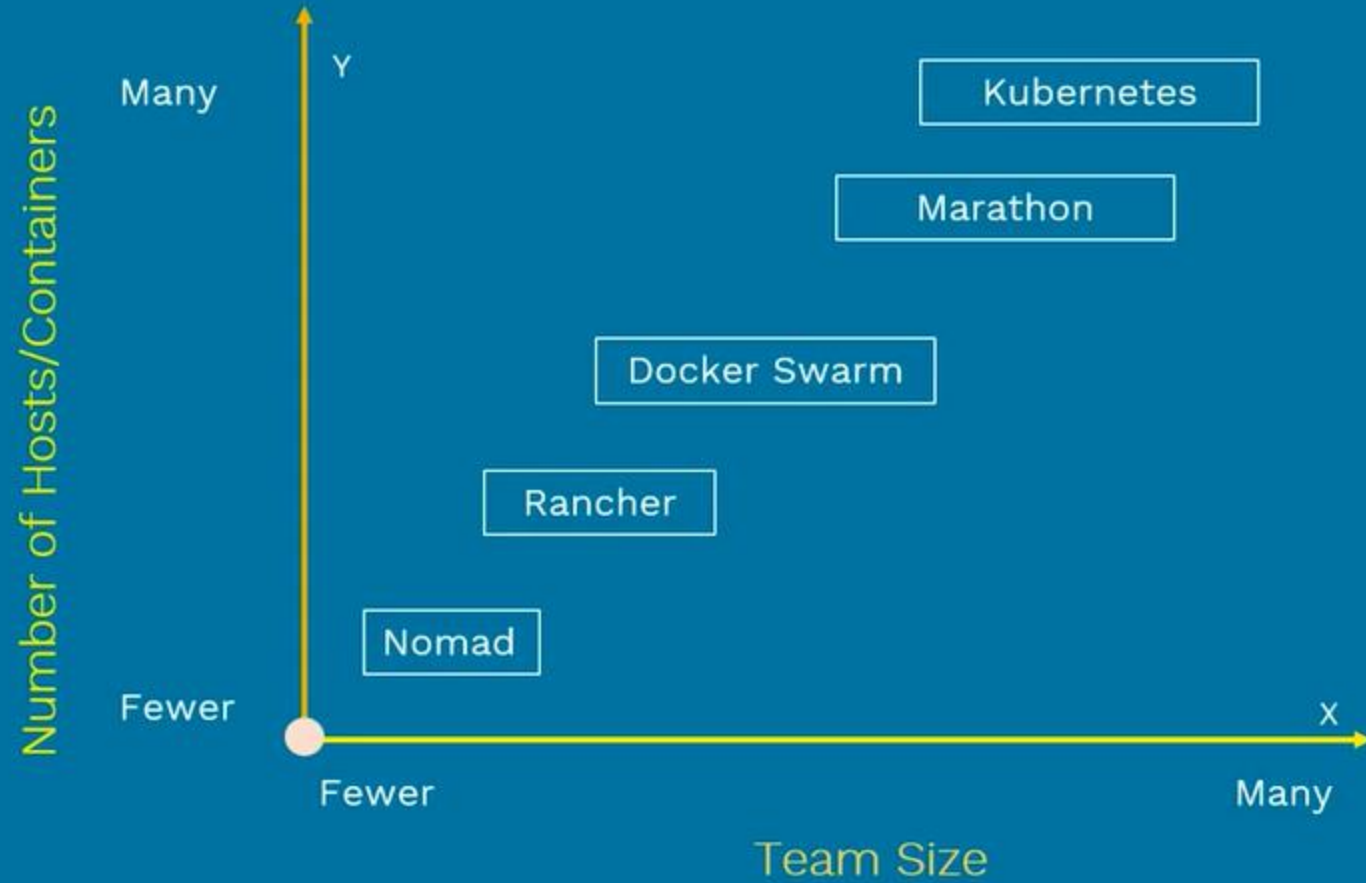


EC2
Container Service



Azure
Container Service

Choosing Orchestrator



Objectives

Part-I:

What is Kubernetes?

History @ Google Inc.

Current status

Part-II:

Why Kubernetes is best COE?

What is Kubernetes?

COE

Manages containerized apps @ large scale

cluster management | scheduling | service discovery monitoring | secrets management
& more

Google → CNCF

History @ Google Inc.

More than Decade

Runs billions of containers every week

Gmail, YouTube, Search ... runs on containers

Borg – Proprietary container manager of Google

Container Orchestration Engine on Google Cloud

Kubernetes

Released Kubernetes v1 around mid 2015

Google donated Kubernetes to CNCF

Latest stable Kubernetes – 1.11.3

Goals and Considerations:

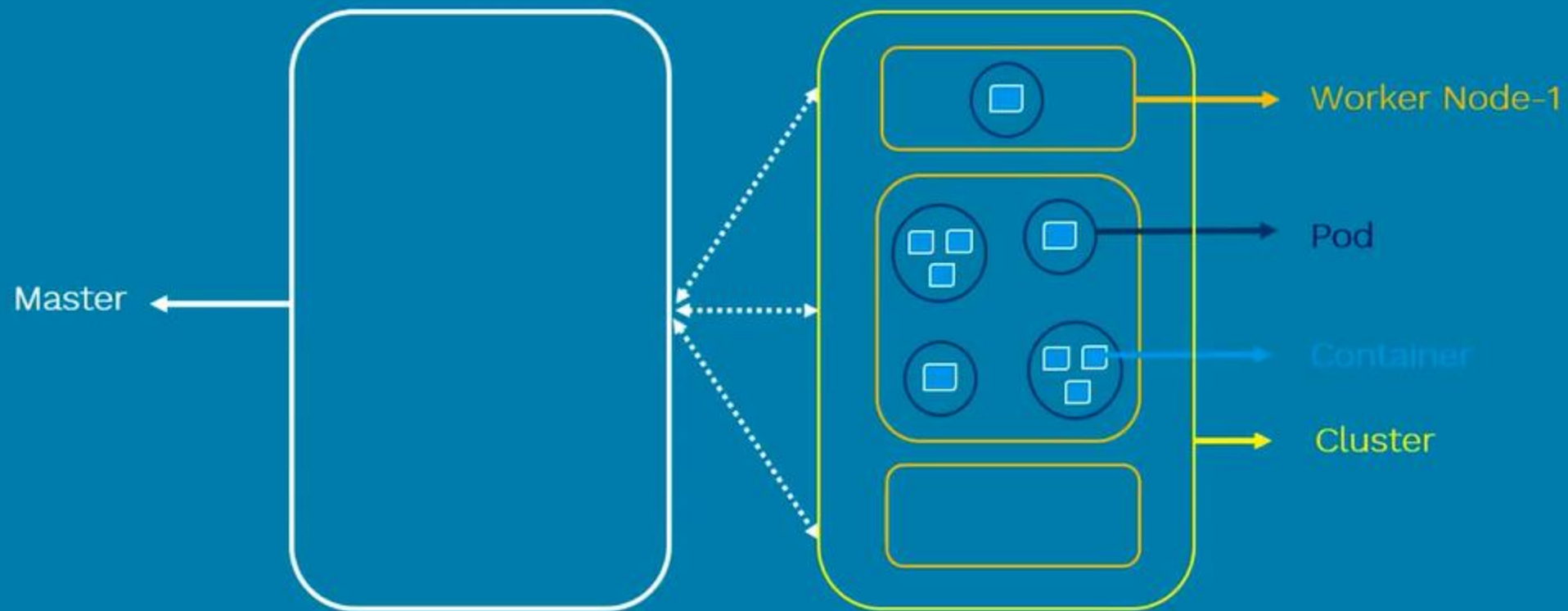
1. Empower application developers
2. Provide standard deployment interface
3. Build on a Modular API code

K8s - Kubernetes

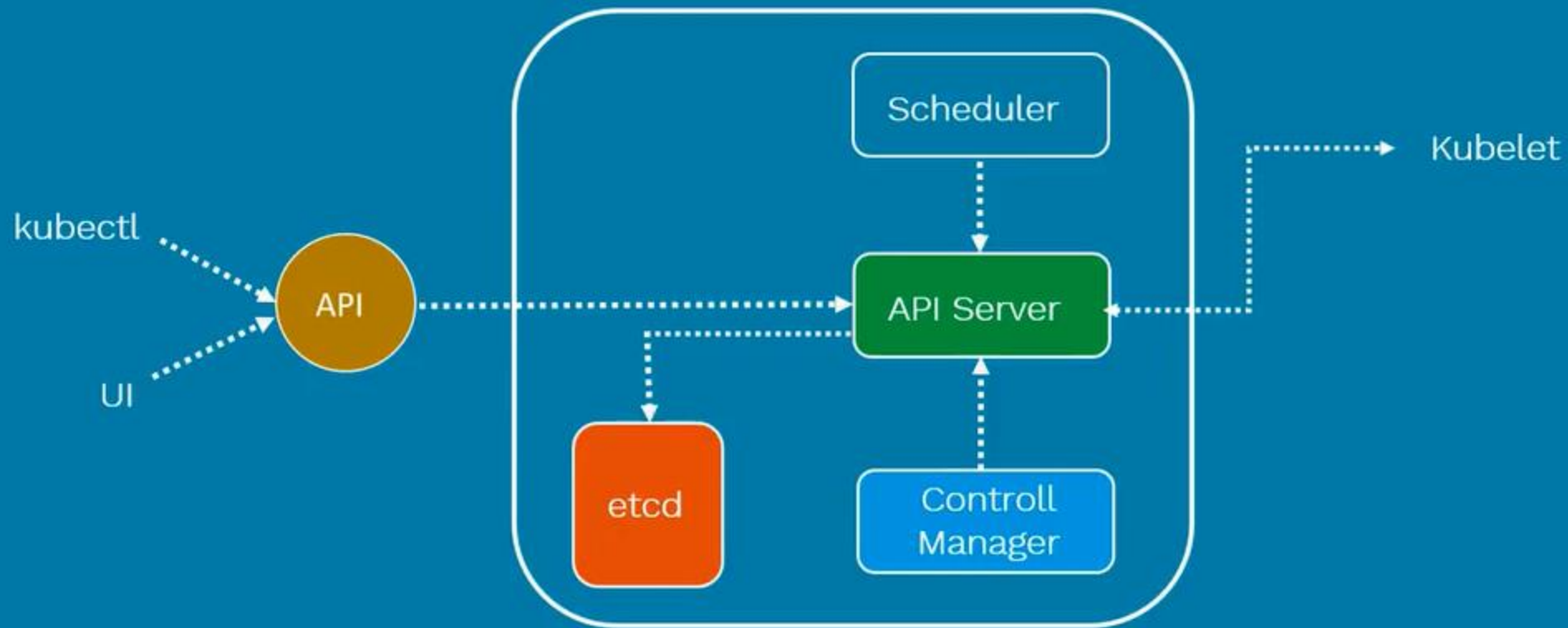
Objectives

1. High-level overview
2. Kubernetes Master
3. Worker node
4. Kubernetes Architecture
5. Terminology

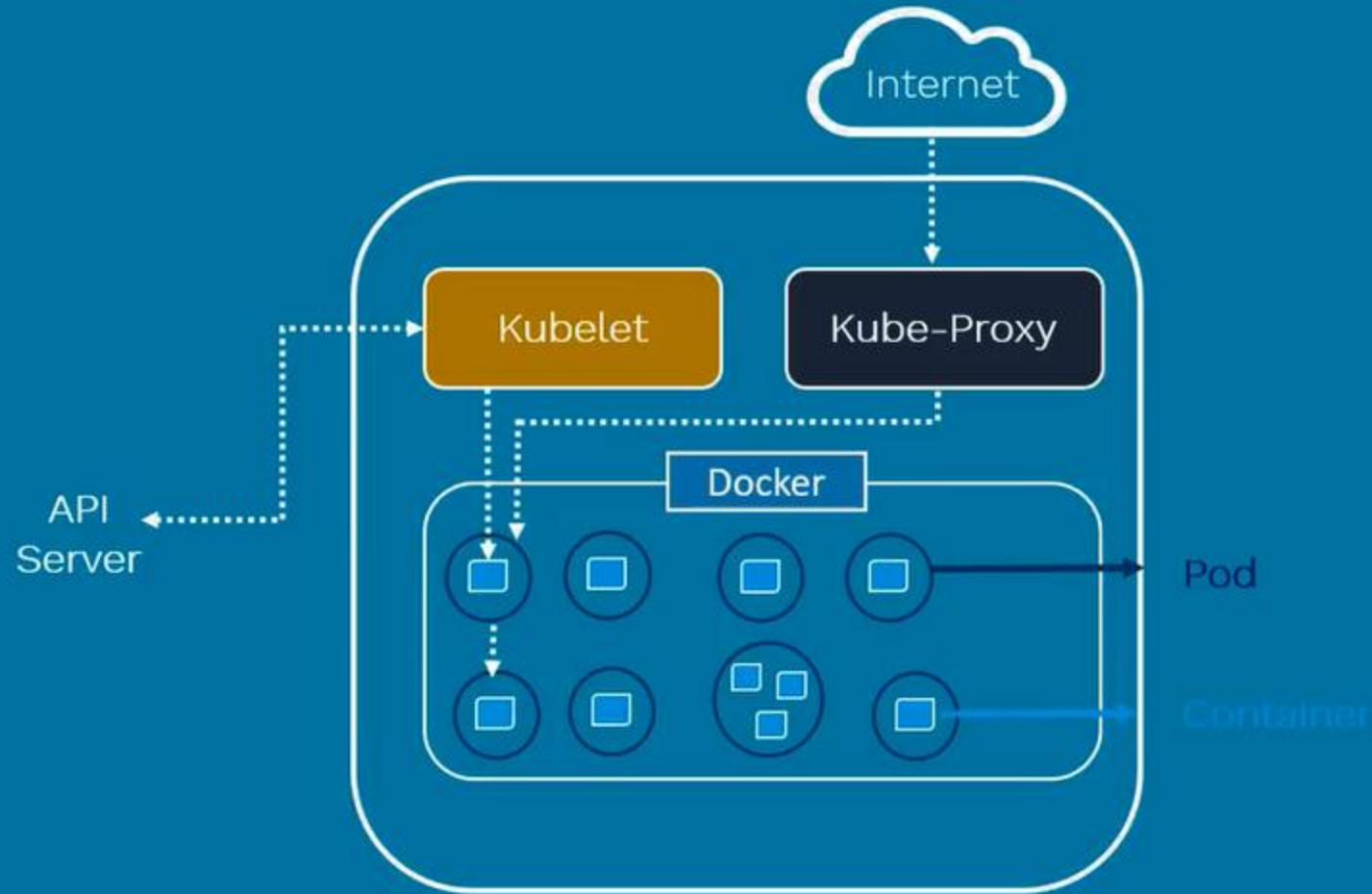
Overview



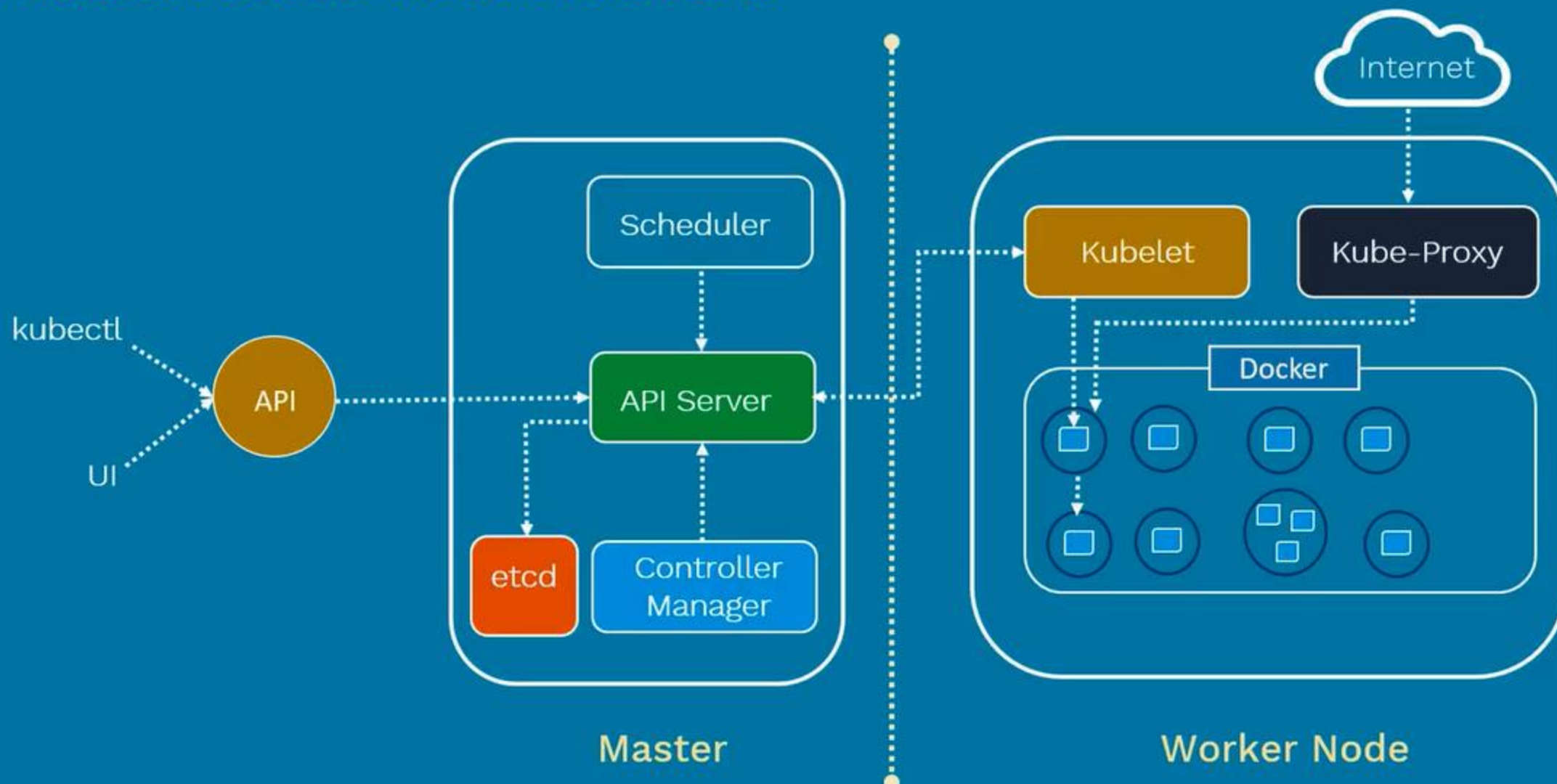
Kubernetes Master



Kubernetes Worker Node



Kubernetes Architecture



Kubernetes Architecture - Terminology

Master

API Server

Scheduler

Controller Manager

ETCD

Worker

Kubelet

Kube-proxy

Pod

Containers

Objectives

1. Different ways of Installing Kubernetes

Different ways of Installing Kubernetes

Play-with-k8s

<https://labs.play-with-k8s.com/>

Minikube

Kubeadm

Objectives

Concept

- Overview of Minikube

Review Demo

- a. Download and Install - VirtualBox, Minikube, & Kubectl
- b. Start Minikube
- c. Test

Minikube

Tool that makes it easy to run Kubernetes locally

Runs all k8s components in single-node inside a VM

Automatically downloads ISO and create VM

Requires Hypervisor(Ex: VirtualBox)

Requires kubectl to manage deployments

Virtual Box



Minikube



Kubectl

Download & Install - Virtual Box, MiniKube & Kubectl

1. Download & Install VirtualBox

2. Download “minikube-windows-amd64”

```
C:\>mkdir Kubernetes
```

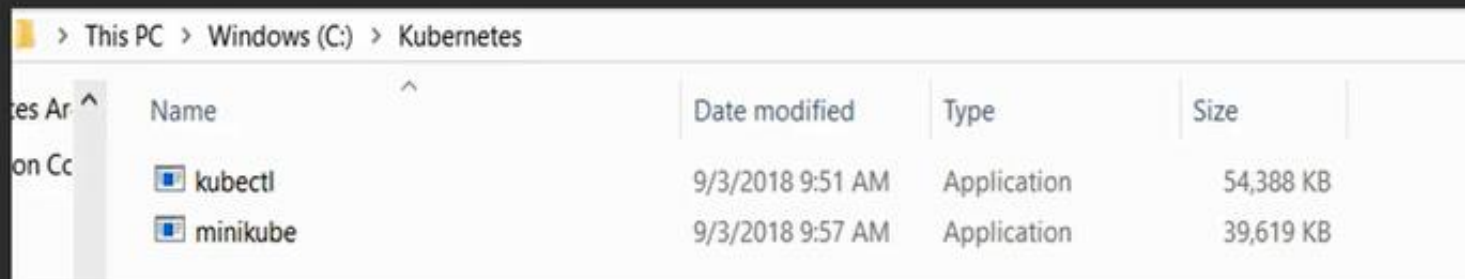
```
C:\>cd Kubernetes
```

<https://github.com/kubernetes/minikube/releases>

NOTE: kubectl and minikube shall appear as shown below in C:\Kubernetes location

3. Download “kubectl”

```
C:\Kubernetes>curl -LO https://storage.googleapis.com/kubernetes-release/release/v1.11.0/bin/windows/amd64/kubectl.exe
```

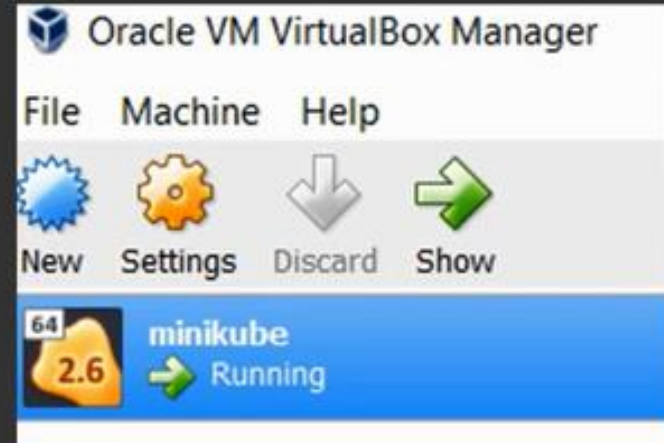


> This PC > Windows (C:) > Kubernetes				
	Name	Date modified	Type	Size
	 kubectl	9/3/2018 9:51 AM	Application	54,388 KB
	 minikube	9/3/2018 9:57 AM	Application	39,619 KB

Minicube (on Windows)

4. Start Minikube

```
C:\Kubernetes>minikube.exe start
Starting local Kubernetes v1.10.0 cluster...
Starting VM...
Downloading Minikube ISO
 160.27 MB / 160.27 MB
[=====]
100.00% 0s
Getting VM IP address...
Moving files into cluster...
Downloading kubelet v1.10.0
Downloading kubeadm v1.10.0
Finished Downloading kubeadm v1.10.0
Finished Downloading kubelet v1.10.0
Setting up certs...
Connecting to cluster...
Setting up kubeconfig...
Starting cluster components...
Kubectl is now configured to use the cluster.
Loading cached images from config file.
```



Minikube: Testing

```
C:\Kubernetes>minikube status
minikube: Running
cluster: Running
kubectl: Correctly Configured: pointing to minikube-vm at 192.168.99.100
```

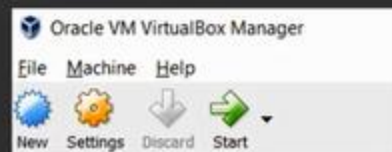
```
C:\Kubernetes>minikube version
minikube version: v0.28.2
```

```
C:\Kubernetes>kubectl run kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1 --port=8080
deployment.apps/kubernetes-bootcamp created
```

```
C:\Kubernetes>kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
kubernetes-bootcamp-5c69669756-b5mk4	1/1	Running	0	48s

```
C:\Kubernetes>minikube stop
Stopping local Kubernetes cluster...
Machine stopped.
```



How do you **install and configure**
multi-node Kubernetes cluster?

Objectives

Concept

- a. Overview of kubeadm
- b. kubeadm commands

Review Demo

- a. Pre-reqs
- b. Installing software applications
- c. Configuring Master
- d. Join Worker Node
- e. Test

kubeadm

Kubeadm helps with installing and configuring
Kubernetes cluster

kubeadm commands

kubeadm **init** //on master

```
kubeadm init [flags]
```

kubeadm **join** //on worker

```
kubeadm join --token [] --discovery-token-ca-cert-hash []
```

kubeadm **token**

```
kubeadm token [create|delete|list|generate] [flags]
```

kubeadm **version**

```
Kubeadm version [flags]
```

kubeadm **upgrade**

```
kubeadm upgrade plan [version] [flags]
```

Configuring Kubernetes by kubeadm

Installing

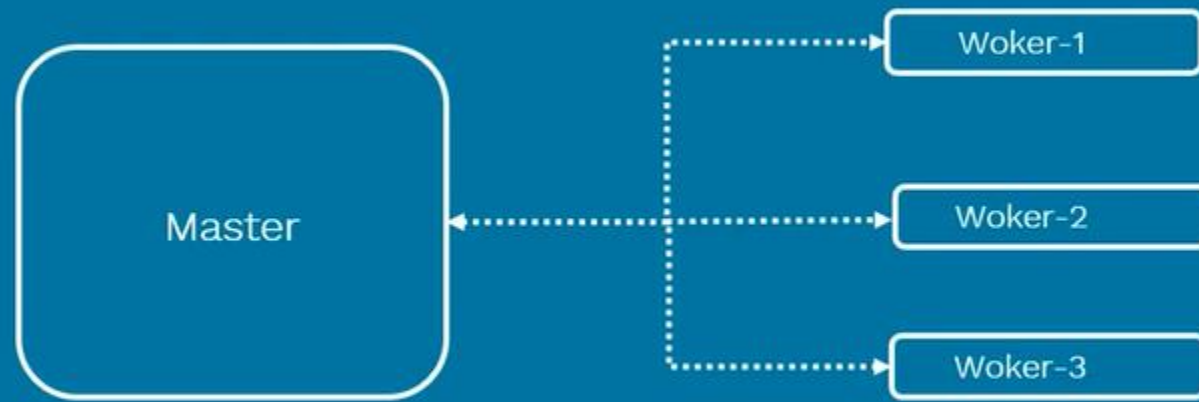
Testing

Pre-reqs

- 3GB or more RAM
- 3 CPU or more
- Full network connectivity among all machines in the cluster.
- Disable SWAP on all nodes
- Disable SELinux on all nodes

Steps

1. Create VMs which are part of k8s cluster (master & worker nodes)
2. Disable SELinux and SWAP on all nodes
3. Install Kubeadm, kubelet, kubectl and Docker on all nodes
 - Start and enable docker and kubelet on all nodes
4. Initialize the master node
5. Configure Pod network
6. Join worker nodes to the cluster



2. Disable SWAP and SELinux on all nodes

Disable SWAP on all nodes:

```
root@master:$ swapoff -a
```

Disable SELinux on all nodes:

```
root@master:$ setenforce 0
root@master:$ sed -i 's/enforcing/disabled/g' /etc/selinux/config
root@master:$ grep disabled /etc/selinux/config | grep -v '#'
SELINUX=disabled
```

Reboot all nodes

3a. Install Docker

Install Docker:

```
root@master:~# yum update -y
root@master:~# yum install -y docker
```

Start and enable Docker:

```
root@master:~# systemctl start docker
root@master:~# systemctl enable docker
root@master:~# systemctl status docker
```

```
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; vendor preset: disabled)
   Active: active (running) since Mon 2018-09-03 13:25:42 UTC; 1s ago
     Docs: http://docs.docker.com
    Main PID: 13833 (dockerd-current)
    CGroup: /system.slice/docker.service
            └─13833 /usr/bin/dockerd-current --add-runtime docker-runc=/usr/libexec/docker/docker-runc-current --default-ru...
              └─13837 /usr/bin/docker-containerd-current -l unix:///var/run/docker/libcontainerd/docker-containerd.sock --met...

Sep 03 13:25:40 master dockerd-current[13833]: time="2018-09-03T13:25:40.795598959Z" level=warning msg="Docker could.."
...
Sep 03 13:25:42 master systemd[1]: Started Docker Application Container Engine.
Sep 03 13:25:42 master dockerd-current[13833]: time="2018-09-03T13:25:42.319356249Z" level=info msg="API listen on /v...sock"
Hint: Some lines were ellipsized, use -l to show in full.
```

3b. Install kubeadm, kubelet, Kubectl

Add Kubernetes Repo:

```
root@master:# cat <<EOF > /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://packages.cloud.google.com/yum/repos/kubernetes-el7-x86_64
enabled=1
gpgcheck=1
repo_gpgcheck=1
gpgkey=https://packages.cloud.google.com/yum/doc/yum-key.gpg
https://packages.cloud.google.com/yum/doc/rpm-package-key.gpg
exclude=kube*
EOF
```

Install kubelet, kubeadm, kubectl and start kubelet:

```
root@master:# yum install -y kubeadm kubelet kubectl --disableexcludes=kubernetes
root@master:# systemctl enable kubelet && systemctl start kubelet
```

```
[root@master ~]# cat <<EOF > /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF

[root@master ~]# sysctl --system
```

NOTE: Applicable only if you are on RHEL/CentOS 7

4. Initialize master node

Only on **master** node:

```
root@master:~# kubectl init --pod-network-cidr=10.240.0.0/16
```

...

Your Kubernetes master has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:

<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

You can now join any number of machines by running the following on each node as root:

```
kubectl join --token a2dc82.7e936a7ba007f01e 10.240.0.1:6443 --discovery-token-ca-cert-hash sha256:30aca9f9c04f829a13c925224b34c47df0a784e9ba94e132a983658a70ee2914
```


5. Configure Pod network

Only on **master** node:

```
root@master:~# kubectl apply -f \
  https://raw.githubusercontent.com/coreos/flannel/v0.9.1/Documentation/kube-flannel.yml
```

```
root@master:~# kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	kube-flannel-ds-3hsdj	2/2	Running	0	1m
kube-system	dummy-2088944543-69gpw	1/1	Running	0	4m
kube-system	etcd-master1	1/1	Running	0	3m
kube-system	kube-apiserver-master1	1/1	Running	0	4m
kube-system	kube-controller-manager-master1	1/1	Running	0	3m
kube-system	kube-discovery-1769846148-n4fjq	1/1	Running	0	4m
kube-system	kube-dns-2924299975-fx7lv	4/4	Running	0	3m
kube-system	kube-proxy-k87q7	1/1	Running	0	3m
kube-system	kube-scheduler-master1	1/1	Running	0	3m

6. Join workers nodes to the cluster

Only on **worker** nodes:

```
root@master:# kubeadm join --token a2dc82.7e936a7ba007f01e 10.240.0.1:6443 --discovery-token-ca-cert-hash  
sha256:30aca9f9c04f829a13c925224b34c47df0a784e9ba94e132a983658a70ee2914
```

```
root@master:# kubeadm token create --print-join-command
```

Testing

```
[root@master ~]# kubectl get no
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	29m	v1.11.2
worker1	Ready	<none>	26m	v1.11.2
worker2	Ready	<none>	24m	v1.11.2
worker3	Ready	<none>	24m	v1.11.2

```
[root@master ~]# kubectl run kubernetes-bootcamp --image=gcr.io/google-samples/kubernetes-bootcamp:v1 --port=8080
```

```
deployment.apps/kubernetes-bootcamp created
```

```
[root@master ~]# kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
kubernetes-bootcamp-69bf88c8c-mcn9p	1/1	Running	0	21s

```
# Note: Execute on all nodes (master & worker)

# Disable Firewall
systemctl stop firewalld
systemctl disable firewalld
# OR - ensure ports [6443 10250] are open

# Disable Swap
swapoff -a
sed -i.bak -r 's/(.+ swap .+)/#\1/' /etc/fstab

# Disable SELinux
setenforce 0
sed -i 's/enforcing/disabled/g' /etc/selinux/config

# Installing Docker | Kubelet | Kubeadm | Kubectl
yum update -y
yum install -y docker kubeadm kubelet kubectl --disableexcludes=kubernetes

# Start and enable docker and kubectl
systemctl enable docker && systemctl start docker
systemctl enable kubelet && systemctl start kubelet

# For CentOS and RHEL
cat <<EOF > /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF

sysctl net.bridge.bridge-nf-call-iptables=1
sysctl net.ipv4.ip_forward=1
sysctl --system
echo "1" > /proc/sys/net/ipv4/ip_forward

# Restart the systemd daemon and the kubelet service with the commands:
systemctl daemon-reload
systemctl restart kubelet
```

In **Virtualization**, basic scheduling unit is **Virtual Machine (VM)**

In **Containerization**, basic scheduling unit is **Container**

In **Kubernetes**, what is basic scheduling unit ?

Objectives

Concept

What is Pod?

Pod deployment

Multi-container

Pod networking

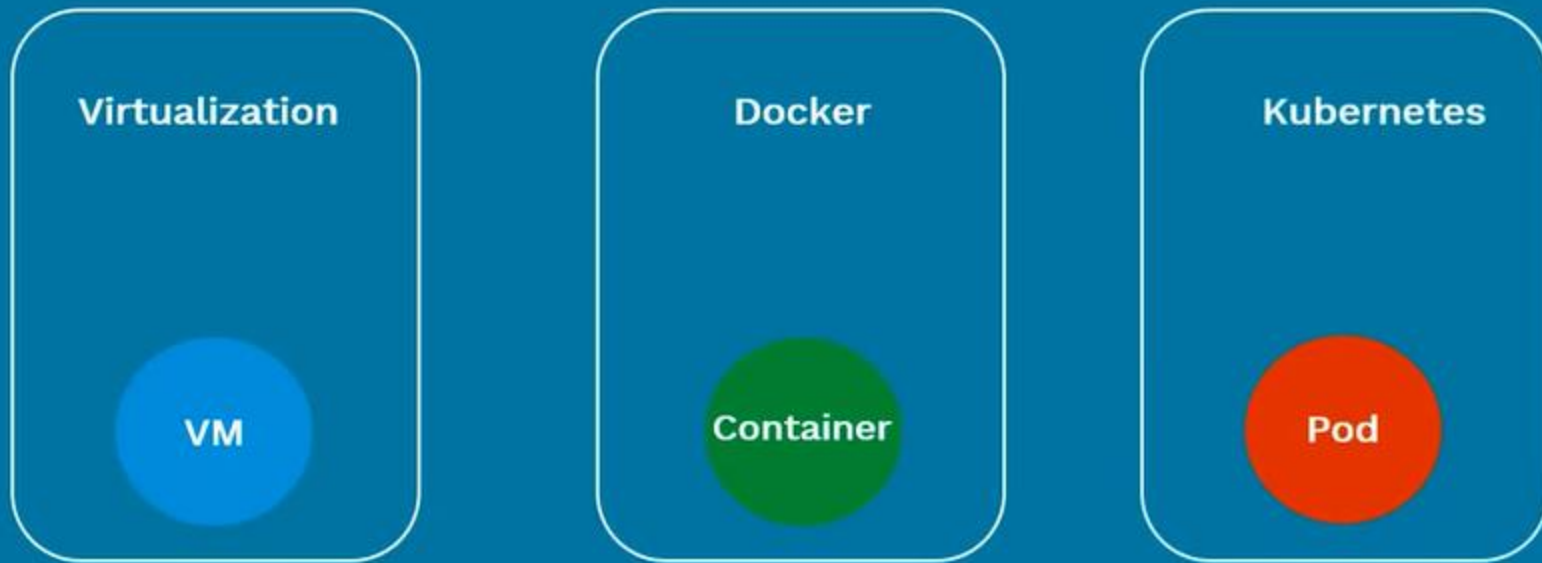
Inter-Pod & Intra-Pod communication

Pod lifecycle

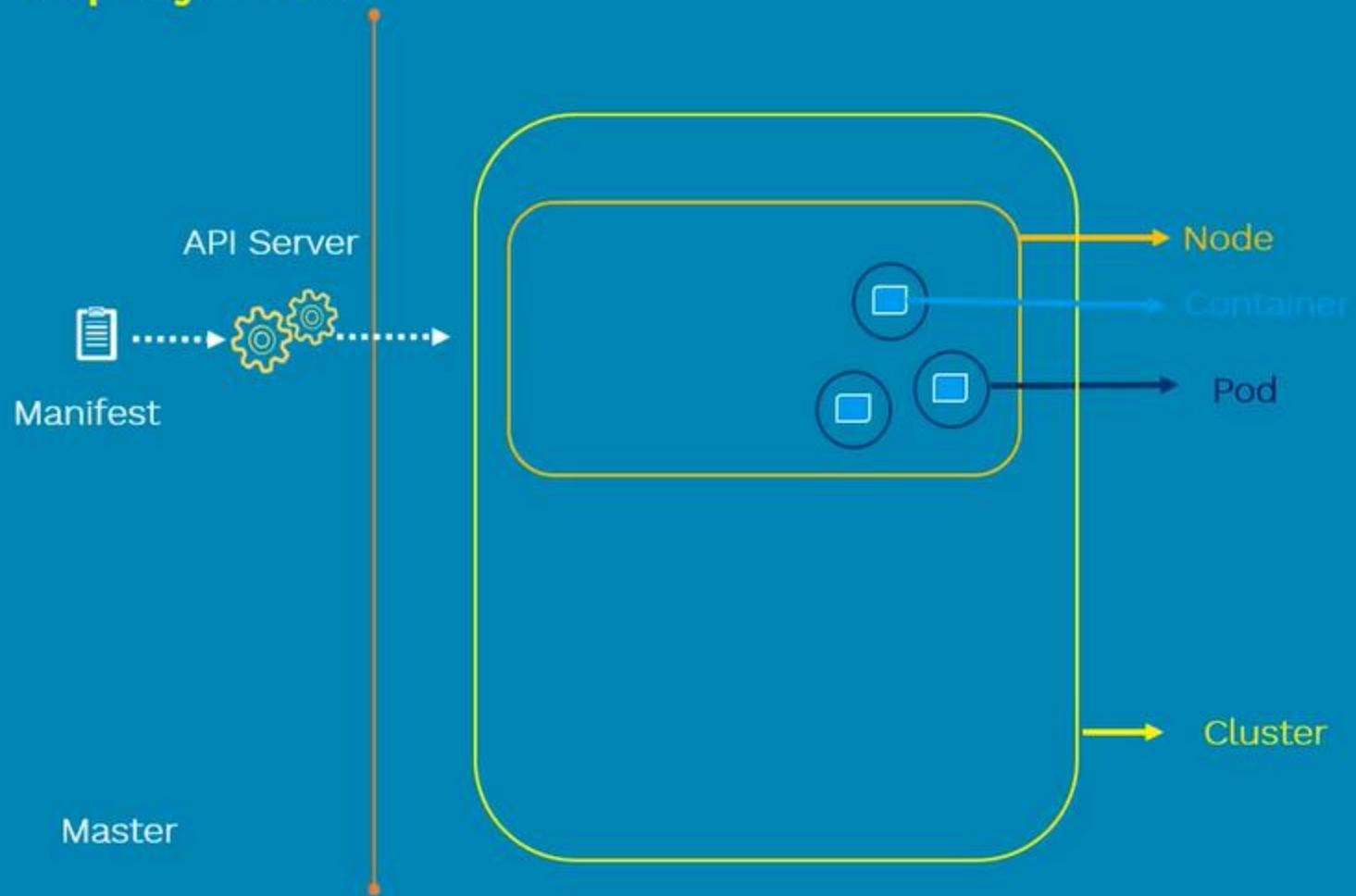
Pod manifest file

What is Pod?

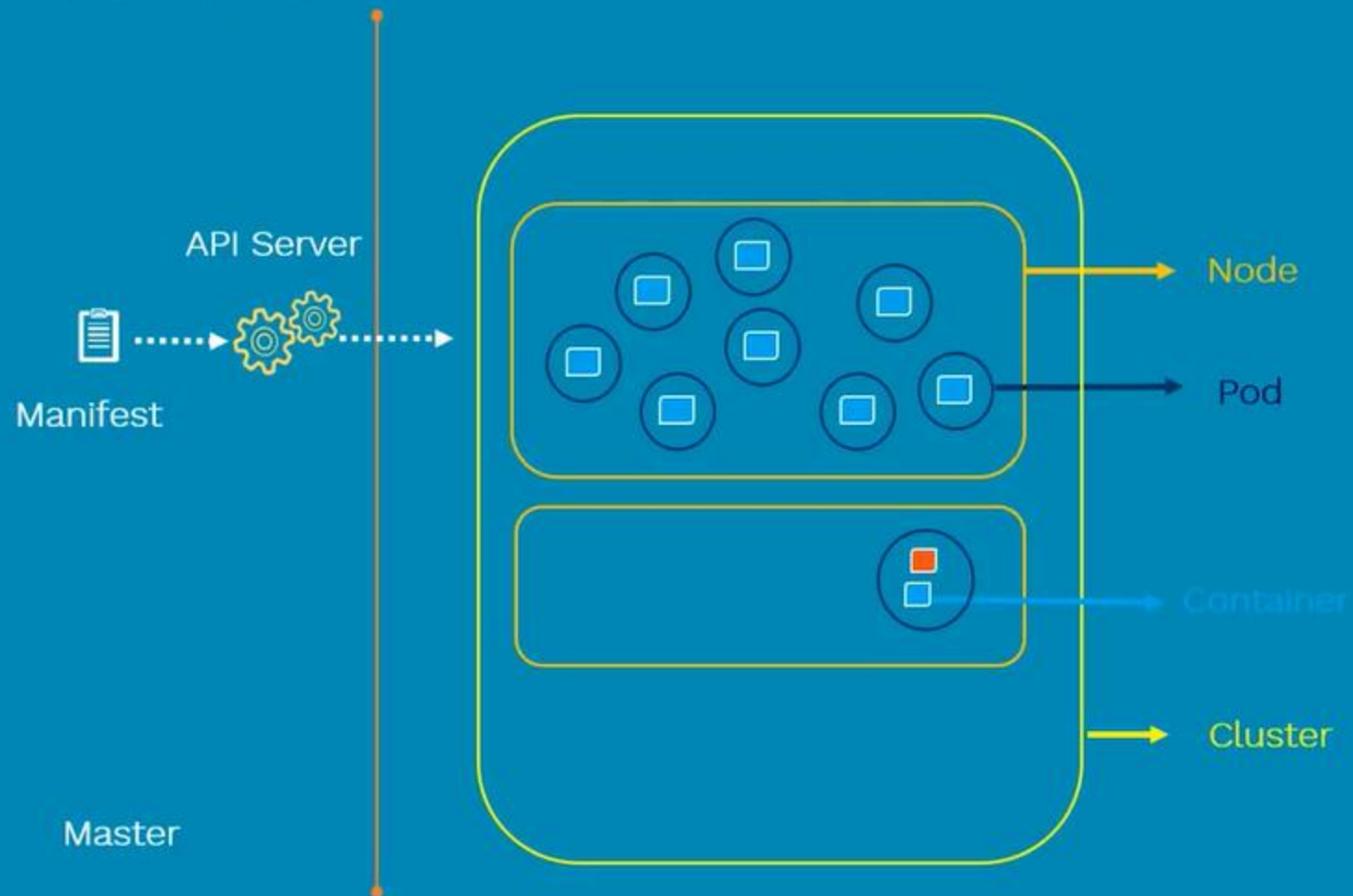
Atomic Unit of Scheduling



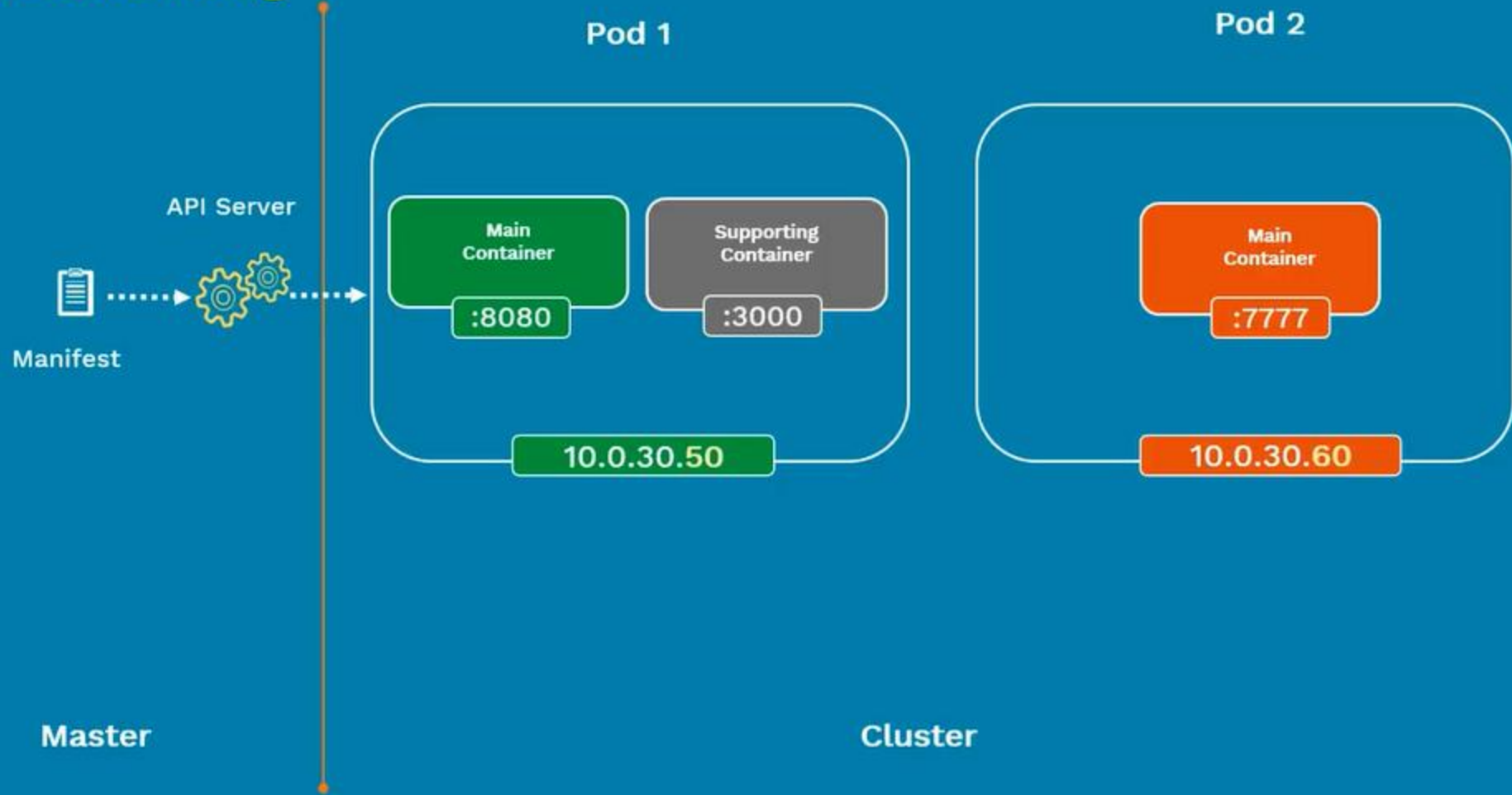
Pod Deployment



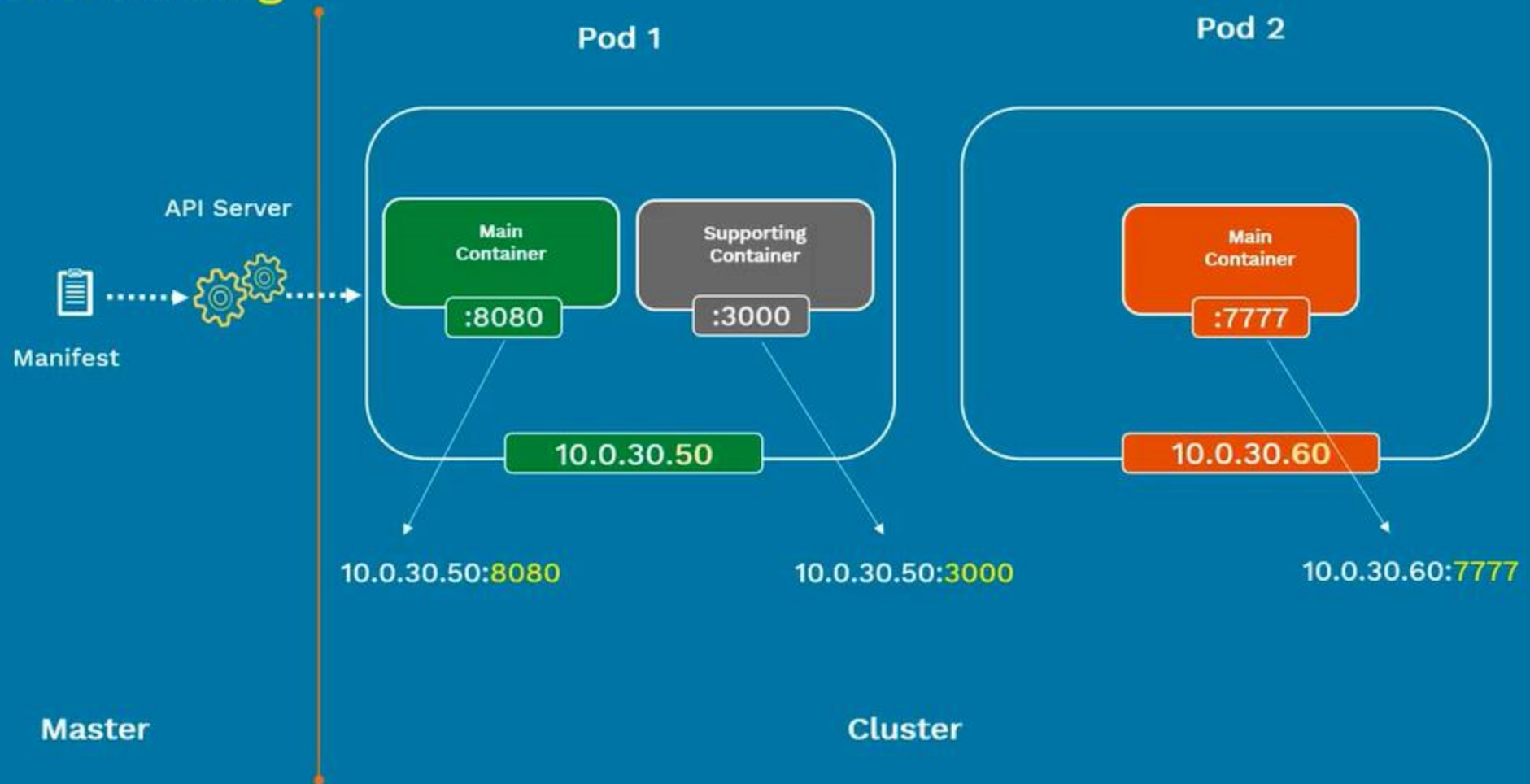
Multi-Container



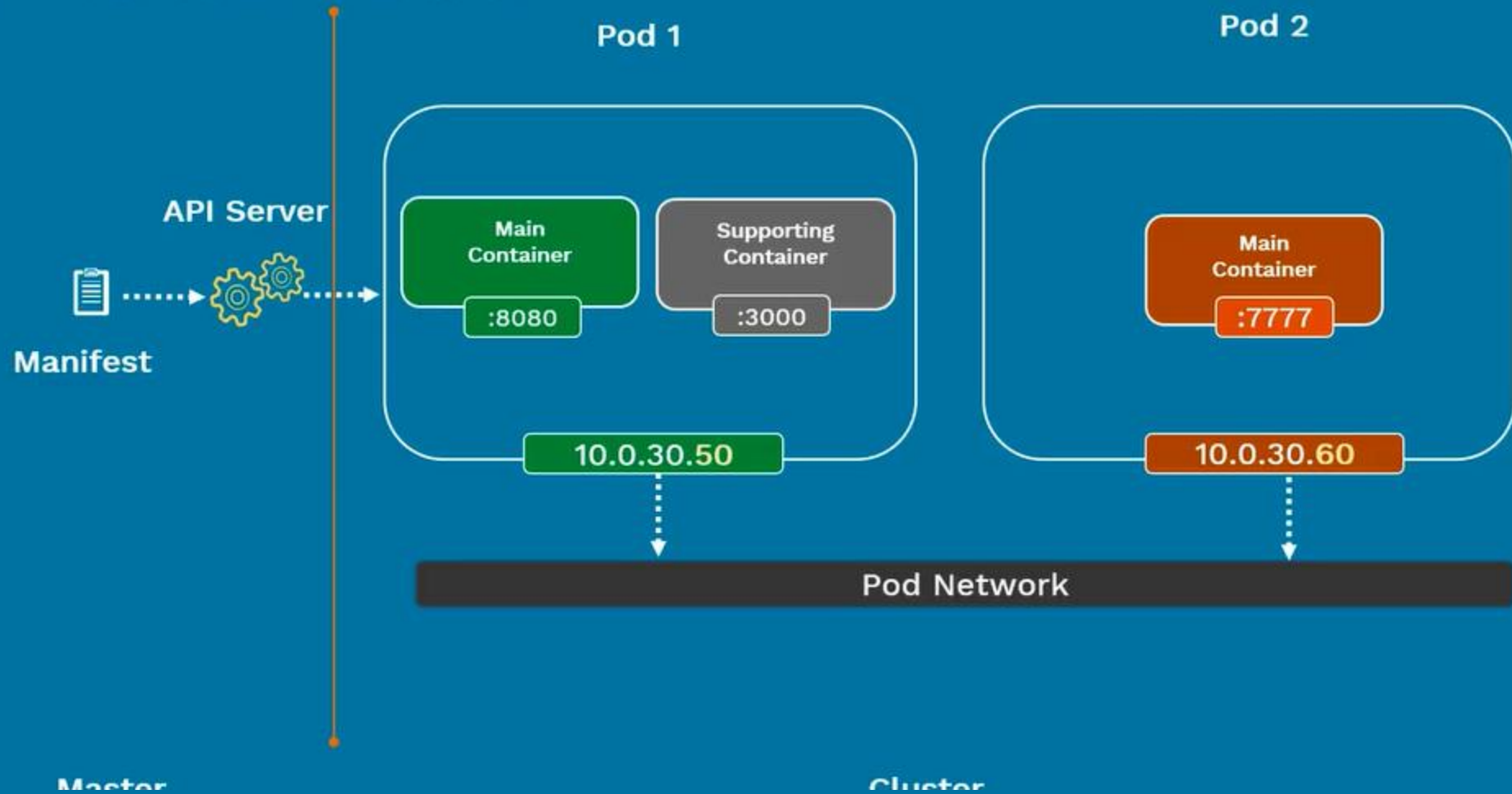
Pod Networking



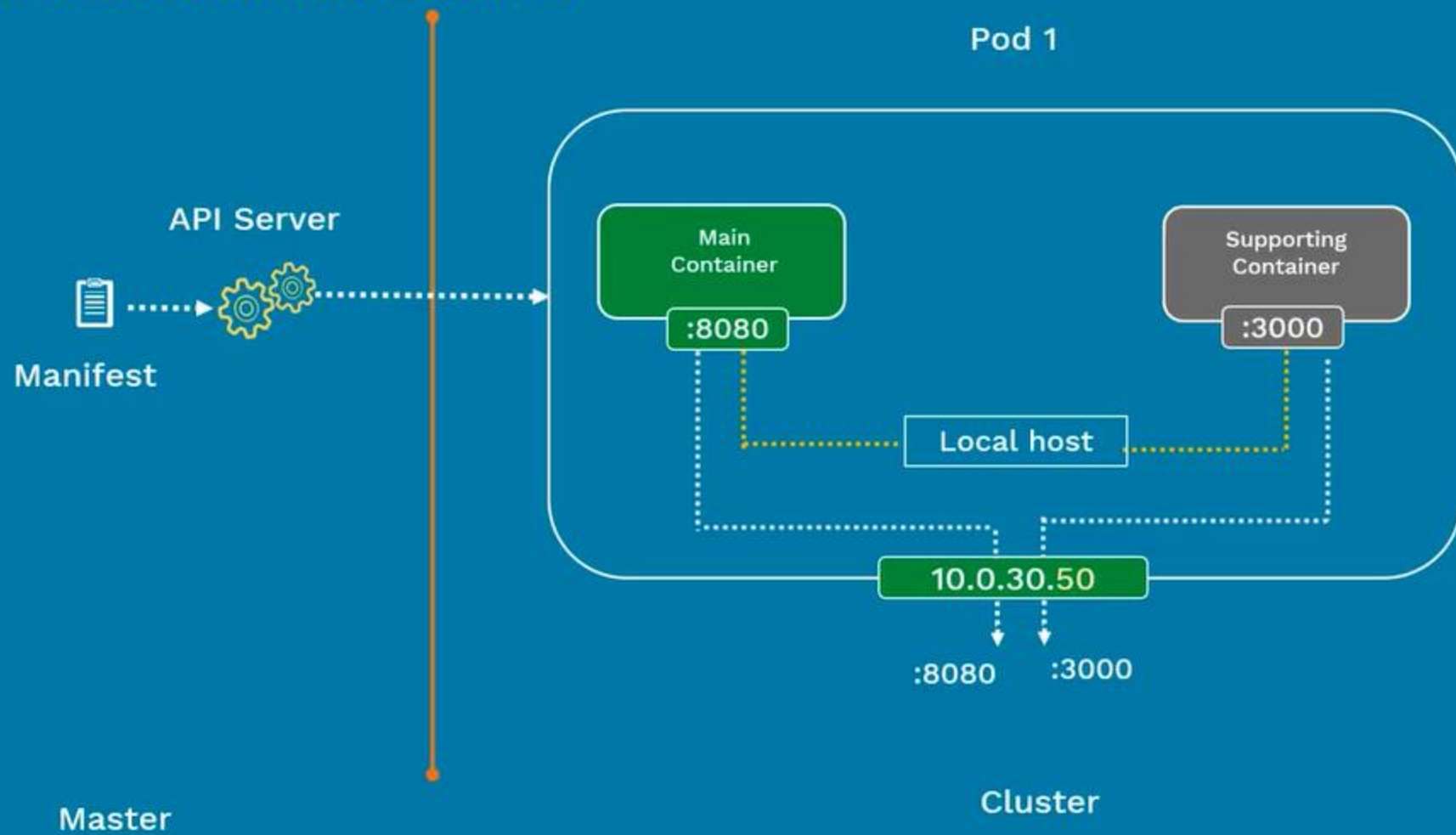
Pod Networking



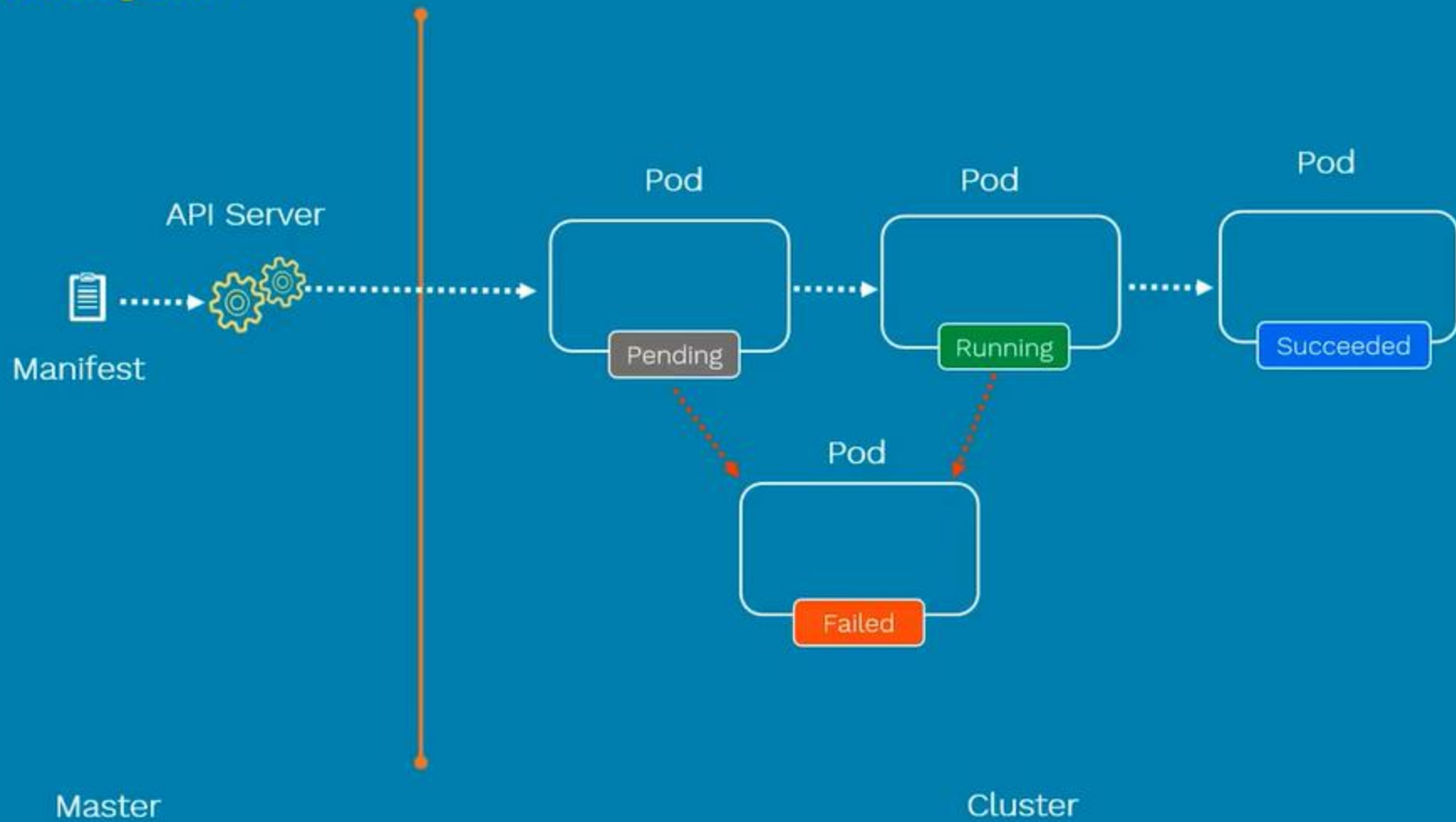
Inter-Pod communication



Intra-Pod communication



Pod lifecycle



Pod - Config

```
# nginx-pod.yaml  
apiVersion: v1  
kind: Pod  
metadata:
```

```
spec:
```

Kind	apiVersion
Pod	v1
ReplicationController	V1
Service	v1
ReplicaSet	apps/v1
Deployment	apps/v1
DaemonSet	apps/v1
Job	batch/v1

Pod - Config

```
# nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
    tier: dev
spec:
  containers:
  - name: nginx-container
    image: nginx
```

Kind	apiVersion
Pod	v1
ReplicationController	V1
Service	v1
ReplicaSet	apps/v1
Deployment	apps/v1
DaemonSet	apps/v1
Job	batch/v1

Alpha

--->

Beta

--->

Stable

Pod – Create & Display

```
[schalla@master ~]$ kubectl create -f nginx-pod.yaml  
pod/nginx-pod created
```

```
[schalla@master ~]$ kubectl get pod  
NAME          READY   STATUS    RESTARTS   AGE  
nginx-pod     1/1     Running   0           2m
```

```
[schalla@master ~]$ kubectl get pod -o wide  
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE  
nginx-pod     1/1     Running   0           8m    10.240.1.26   node1    <none>
```

```
[schalla@master ~]$ kubectl get pod nginx-pod -o yaml  
apiVersion: v1  
items:  
- apiVersion: v1  
.....
```

Pod - Describe

```
[schalla@master ~]$ kubectl describe pod nginx-pod # Display all details and events of a pod
```

```
Name:          nginx-pod
Namespace:      default
Priority:        0
PriorityClassName: <none>
Node:           node1/10.128.0.5
Start Time:     Tue, 28 Aug 2018 07:06:55 +0000
Labels:         app=nginx
                tier=dev
Annotations:    <none>
Status:         Running
IP:             10.240.1.26
```

```
Containers:
  nginx-container:
```

```
.....
```

```
Events:
```

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	Scheduled	14m	default-scheduler	Successfully assigned default/nginx-pod to node1
Normal	Pulling	14m	kubelet, node1	pulling image "nginx"
Normal	Pulled	14m	kubelet, node1	Successfully pulled image "nginx"
Normal	Created	14m	kubelet, node1	Created container
Normal	Started	14m	kubelet, node1	Started container

Pod - Testing

```
[schalla@master ~]$ ping 10.240.1.26          #Pinging Container IP from master
PING 10.240.1.26 (10.240.1.26) 56(84) bytes of data.
64 bytes from 10.240.1.26: icmp_seq=1 ttl=63 time=0.387 ms
64 bytes from 10.240.1.26: icmp_seq=2 ttl=63 time=0.361 ms
64 bytes from 10.240.1.26: icmp_seq=3 ttl=63 time=0.246 ms
^C
--- 10.240.1.26 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2000ms
rtt min/avg/max/mdev = 0.246/0.331/0.387/0.063 ms
```

```
[schalla@master ~]$ kubectl exec -it nginx-pod -- /bin/sh #Getting a shell to running cont
# hostname
nginx-pod
# exit
```

```
[schalla@master ~]$ kubectl delete pod nginx-pod
pod "nginx-pod" deleted
```


How can you ensure there are at least 3 Pod instances (example)
are always available and running at point in time?

Objectives

Concept

- a. Replication Controller Overview
- b. Advantages
- c. Manifest file

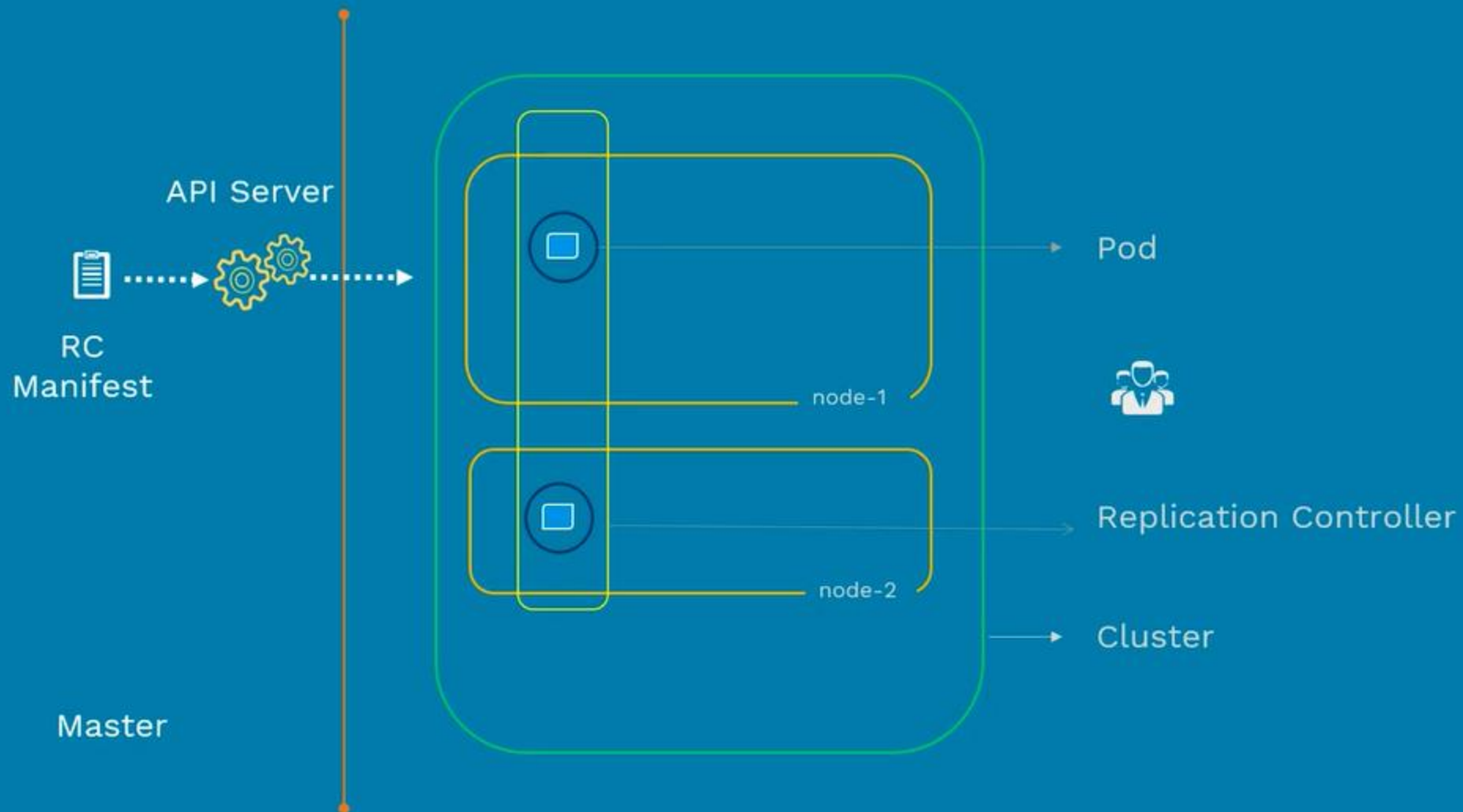
Review Demo

- a. Deploy application with Replication Controller
- b. Display and validate
- c. Scale up and Scale down the application
- d. Clean up

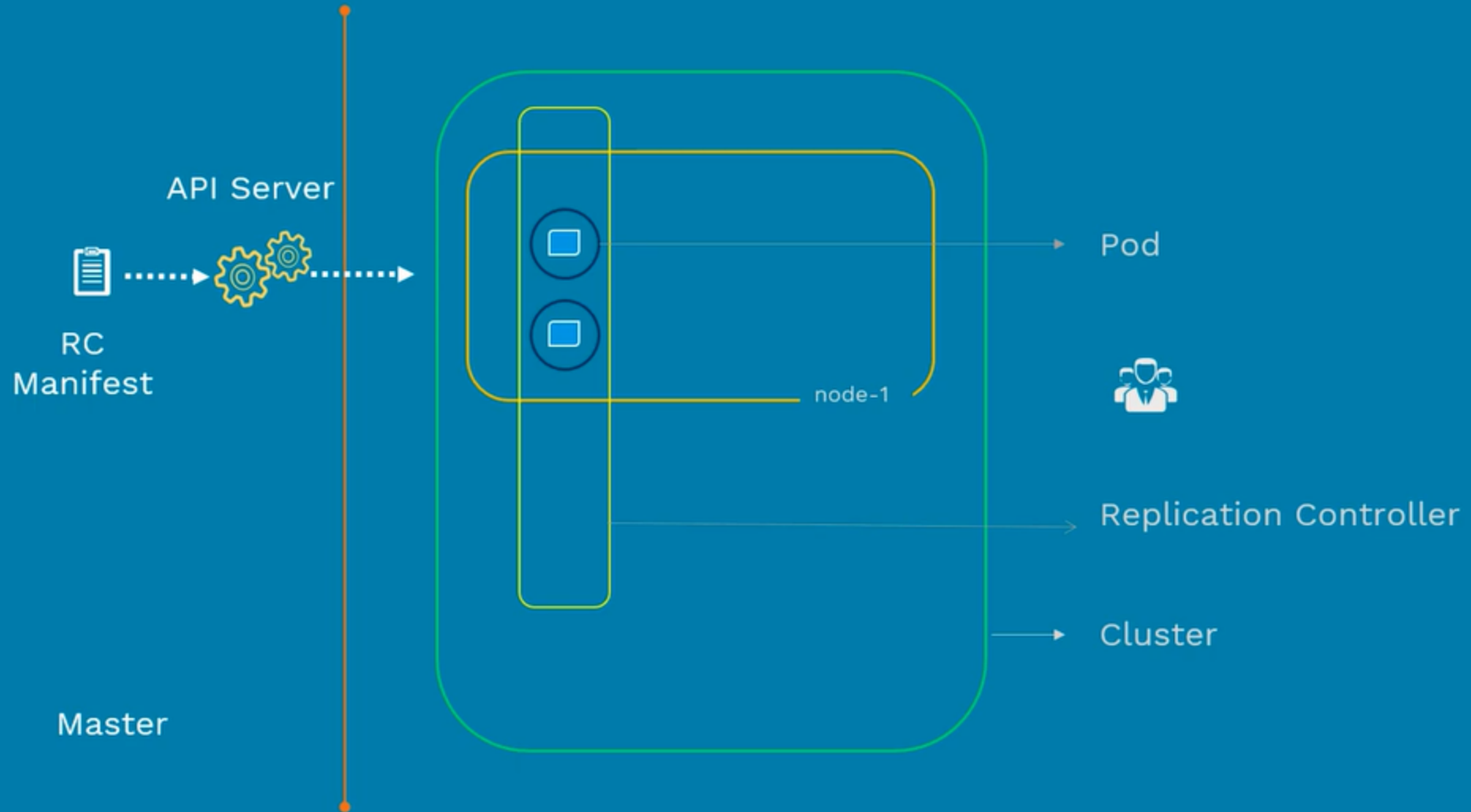
Replication Controller

- Ensures that a specified number of pods are running at any time
 - a. If there are excess Pods, they get killed and vice versa
 - b. New Pods are launched when they get fail, get deleted or terminated
- Replication Controllers and Pods are associated with “ labels ”
- Creating a “rc” with count of 1 ensure that a pod is always available

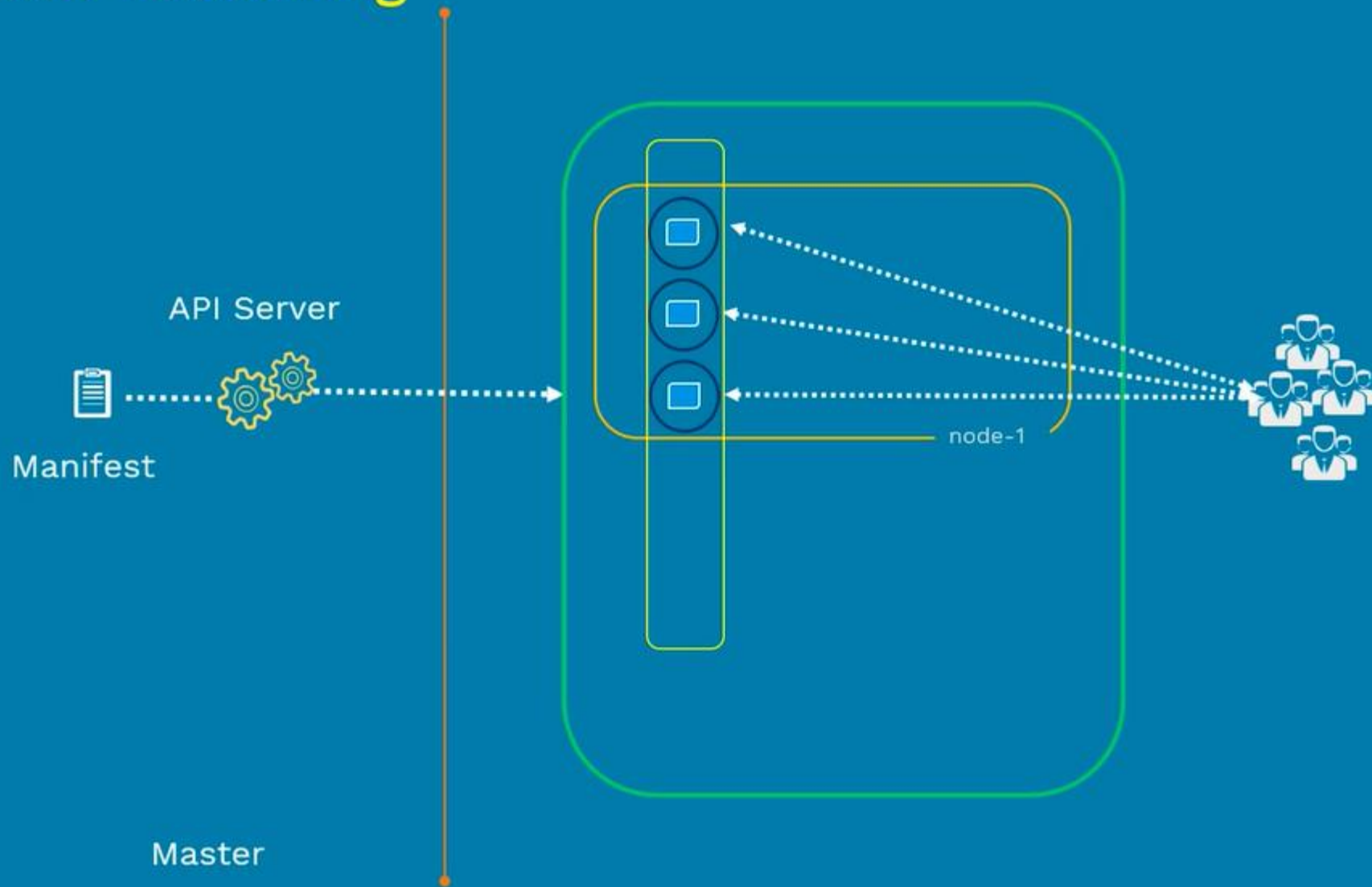
High Availability



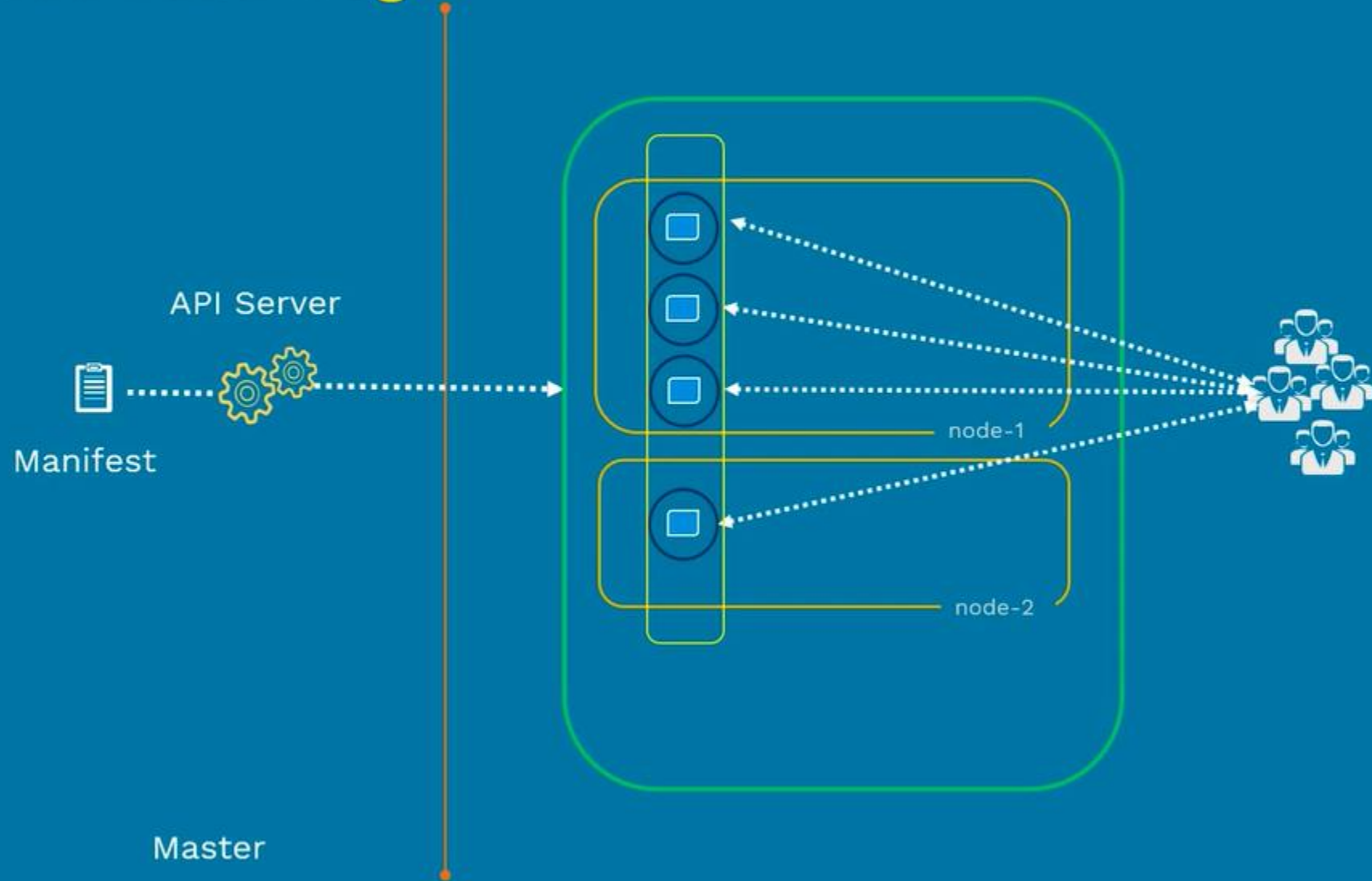
High Availability



Load Balancing



Load Balancing



One last thing

Replication Controller - OLD



ReplicaSet - NEW

Review

Demo

- | | | |
|----------------------------|--|-----------------------------|
| a. Manifest file | | d. Test – Node fails |
| b. Deploy app using RC | | e. Test – Scale up |
| c. Display and validate RC | | f. Test – Scale down |

Replication Controller – Config

```
# nginx-rc.yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx-rc
spec:
  replicas: 3
  selector:
    app: nginx-app
  template:
    metadata:
      name: nginx-pod
      labels:
        app: nginx-app
    spec:
      containers:
      - name: nginx-container
        image: nginx
        ports:
        - containerPort: 80
```

Pod def

Replication Controller – Create & Display

```
[srinath@master ~]$ kubectl create -f nginx-rc.yaml  
replicationcontroller/nginx-rc created
```

```
[srinath@master ~]$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-rc-62vpv	1/1	Running	0	6m
nginx-rc-fk67w	1/1	Running	0	6m
nginx-rc-qk4ph	1/1	Running	0	6m

```
[srinath@master ~]$ kubectl get po -l app=nginx-app
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-rc-62vpv	1/1	Running	0	4m
nginx-rc-fk67w	1/1	Running	0	4m
nginx-rc-qk4ph	1/1	Running	0	4m

Replication Controller – Describe

```
[srinath@master ~]$ kubectl describe rc nginx-rc
```

```
Name:          nginx-rc
Namespace:     default
Selector:      app=nginx-app
Labels:        app=nginx-app
Annotations:   <none>
Replicas:      3 current / 3 desired
Pods Status:   3 Running / 0 Waiting / 0 Succeeded / 0 Failed
```

Pod Template:

```
Labels: app=nginx-app
```

Containers:

nginx-container:

```
Image:          nginx
Port:           80/TCP
Host Port:      0/TCP
Environment:    <none>
Mounts:         <none>
Volumes:        <none>
```

Events:

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	SuccessfulCreate	25m	replication-controller	Created pod: nginx-rc-62vpv
Normal	SuccessfulCreate	25m	replication-controller	Created pod: nginx-rc-fk67w
Normal	SuccessfulCreate	25m	replication-controller	Created pod: nginx-rc-qk4ph

Replication Controller – Node Fail

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
nginx-rc-5lb6b	1/1	Running	0	16s	10.240.1.32	node1	<none>
nginx-rc-l64js	1/1	Running	0	16s	10.240.1.33	node1	<none>
nginx-rc-qrvv7	1/1	Running	0	16s	10.240.2.28	node2	<none>

```
[srinath@master ~]$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	1d	v1.11.2
node1	Ready	<none>	1d	v1.11.2
node2	NotReady	<none>	1d	v1.11.2

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
nginx-rc-5lb6b	1/1	Running	0	11m	10.240.1.32	node1	<none>
nginx-rc-l64js	1/1	Running	0	11m	10.240.1.33	node1	<none>
nginx-rc-mtwzs	1/1	Running	0	2m	10.240.1.34	node1	<none>
nginx-rc-qrvv7	1/1	Unknown	0	11m	10.240.2.28	node2	<none>

Replication Controller – Scaling up

```
[srinath@master ~]$ kubectl scale rc nginx-rc --replicas=5
replicationcontroller/nginx-rc scaled
```

```
[srinath@master ~]$ kubectl get rc nginx-rc
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-rc	5	5	5	9m

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-rc-2x2kg	1/1	Running	0	36m	10.240.1.173	node1	<none>	
nginx-rc-7kvhl	1/1	Running	0	36m	10.240.2.3	node2	<none>	
nginx-rc-g7mwq	1/1	Running	0	33m	10.240.2.42	node2	<none>	
nginx-rc-jgt28	1/1	Running	0	33m	10.240.1.3	node1	<none>	
nginx-rc-wvmrx	1/1	Running	0	36m	10.240.2.4	node2	<none>	

Replication Controller – Scaling down

```
[srinath@master ~]$ kubectl scale rc nginx-rc --replicas=3
replicationcontroller/nginx-rc scaled
```

```
[srinath@master ~]$ kubectl get rc nginx-rc
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-rc	3	3	3	45m

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-rc-2x2kg	1/1	Running	0	36m	10.240.1.173	node1	<none>	
nginx-rc-jgt28	1/1	Running	0	33m	10.240.1.3	node1	<none>	
nginx-rc-wvmrx	1/1	Running	0	36m	10.240.2.4	node2	<none>	

Replication Controller – Delete

```
[srinath@master ~]$ kubectl delete -f nginx-rc.yaml #kubectl delete rc nginx-rc  
replicationcontroller "nginx-rc" deleted
```

```
[srinath@master ~]$ kubectl get rc  
No resources found.
```

```
[srinath@master ~]$ kubectl get po -l app=nginx-app  
No resources found.
```

Objectives

Concept

- a. Overview - ReplicaSet
- b. Labels & Selectors
- c. Equality-based and Set-based selectors

Review Demo

- a. Manifest file – ReplicaSet
- b. Deploy application with ReplicaSet
- c. Display and validate
- d. Test Cases
- e. Clean up

ReplicaSet

- Ensures that a specified number of pods are running at any time
 - a. If there are excess Pods, they get killed and vice versa
 - b. New Pods are launched when they get fail, get deleted or terminated
- ReplicaSet and Pods are associated with “ labels ”

ReplicaSet vs. Replication Controller

ReplicaSet is Next-generation Replication Controller

ReplicaSet
↓
Set-based Selectors

Replication Controller
↓
Equality-based Selectors

Labels & Selectors

Pods

Labels

```
#Pod-Spec
apiVersion: v1
kind: pod
metadata:
  name: nginx-pod
  labels:
    app: guestbook
    tier: frontend
    env: dev
spec:
  replicas: 5
  . . . . .
```

Controllers & Services

Selectors



Equality-based

Operators:

`=` `==` `!=`

Examples:

```
environment = production
tier != frontend
```

Command line

```
$ kubectl get pods -l environment=production
```

In manifest:

```
...
selector:
  environment: production
  tier: frontend
...
```

Set-based

Operators:

`in` `notin` `exists`

Examples:

```
environment in (production, qa)
tier notin (frontend, backend)
```

Command line

```
$ kubectl get pods -l 'environment in (production)
```

In manifest:

```
...
selector:
  matchExpressions:
    - {key: environment, operator: In, values: [prod, qa]}
    - {key: tier, operator: NotIn, values: [frontend, backend]}
...
```

ReplicaSet– Manifest file

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-app
    matchExpressions:
      - {key: tier, operator: In, values: [frontend]}
  template:
    metadata:
      name: nginx-pod
    labels:
      app: nginx-app
      tier: frontend
    spec:
      containers:
        - name: nginx-container
          image: nginx
          ports:
            - containerPort: 80
```

The diagram illustrates the relationship between the ReplicaSet selector and the Pod template labels. A box around the `matchLabels: app: nginx-app` in the selector has a line that connects to the `app: nginx-app` label in the Pod template. Another box around the `matchExpressions: - {key: tier, operator: In, values: [frontend]}` in the selector has a line that connects to the `tier: frontend` label in the Pod template. This shows how the ReplicaSet ensures that only Pods with these specific labels are managed by it.

ReplicaSet– Create & Display

```
[srinath@master ~]$ kubectl create -f nginx-rs.yaml  
replicaset/nginx-rs created
```

```
[srinath@master ~]$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-rs-62vpv	1/1	Running	0	6m
nginx-rs-fk67w	1/1	Running	0	6m
nginx-rs-qk4ph	1/1	Running	0	6m

```
[srinath@master ~]$ kubectl get po -l tier=frontend
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-rs-62vpv	1/1	Running	0	4m
nginx-rs-fk67w	1/1	Running	0	4m
nginx-rs-qk4ph	1/1	Running	0	4m

ReplicaSet– Describe

```
srinath@master:$ kubectl get rs nginx-rs -o wide
```

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
nginx-rs	3	3	3	14s	nginx-container	nginx	app=nginx-app,tier in (frontend)

```
srinath@master:$ kubectl describe rs nginx-rs
```

```
Name:      nginx-rs
Namespace: default
Selector:  app=nginx-app
Labels:    app=nginx-app
Annotations: <none>
Replicas:  3 current / 3 desired
Pods Status: 3 Running / 0 Waiting / 0 Succeeded / 0 Failed
```

Pod Template:

Labels: app=nginx-app

Containers:

nginx-container:

Image: nginx

Port: 80/TCP

Environment: <none>

Mounts: <none>

Volumes: <none>

Events:

Type	Reason	Age	From	Message
Normal	SuccessfulCreate	8m	replicaset-controller	Created pod: nginx-rs-8vcfq
Normal	SuccessfulCreate	8m	replicaset-controller	Created pod: nginx-rs-l2t5z

ReplicaSet – Scheduling

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED
nginx-rc-5lb6b	1/1	Running	0	16s	10.240.1.32	node1	<none>
nginx-rc-l64js	1/1	Running	0	16s	10.240.1.33	node1	<none>
nginx-rc-qrqv7	1/1	Running	0	16s	10.240.2.28	node2	<none>

```
[srinath@master ~]$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	1d	v1.11.2
node1	Ready	<none>	1d	v1.11.2
node2	NotReady	<none>	1d	v1.11.2

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NC
nginx-rc-5lb6b	1/1	Running	0	11m	10.240.1.32	node1	<none>	
nginx-rc-l64js	1/1	Running	0	11m	10.240.1.33	node1	<none>	
nginx-rc-mtwzs	1/1	Running	0	2m	10.240.1.34	node1	<none>	
nginx-rc-qrqv7	1/1	Unknown	0	11m	10.240.2.28	node2	<none>	

ReplicaSet – Scaling up

```
[srinath@master ~]$ kubectl scale rs nginx-rs --replicas=5
replicationcontroller/nginx-rc scaled
```

```
[srinath@master ~]$ kubectl get rs nginx-rc
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-rc	5	5	5	9m

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-rc-2x2kg	1/1	Running	0	36m	10.240.1.173	node1	<none>	
nginx-rc-7kvhl	1/1	Running	0	36m	10.240.2.3	node2	<none>	
nginx-rc-g7mwq	1/1	Running	0	33m	10.240.2.42	node2	<none>	
nginx-rc-jgt28	1/1	Running	0	33m	10.240.1.3	node1	<none>	
nginx-rc-wvmrx	1/1	Running	0	36m	10.240.2.4	node2	<none>	

ReplicaSet – Scaling down

```
[srinath@master ~]$ kubectl scale rs nginx-rs --replicas=3
replicationcontroller/nginx-rc scaled
```

```
[srinath@master ~]$ kubectl get rs nginx-rs
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-rc	3	3	3	45m

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE
nginx-rc-2x2kg	1/1	Running	0	36m	10.240.1.173	node1	<none>	
nginx-rc-jgt28	1/1	Running	0	33m	10.240.1.3	node1	<none>	
nginx-rc-wvmrx	1/1	Running	0	36m	10.240.2.4	node2	<none>	

ReplicaSet – Delete

```
[srinath@master ~]$ kubectl delete -f nginx-rc.yaml #kubectl delete rc nginx-rc  
replicationcontroller "nginx-rc" deleted
```

```
[srinath@master ~]$ kubectl get po -l app=nginx-app  
No resources found.
```

Imagine, you are upgrading application from v1 to v2

Upgrade with zero downtime?

Upgrade sequentially, one after the other?

Pause and resume upgrade process?

Rollback upgrade to previous stable release

Objectives

Concept

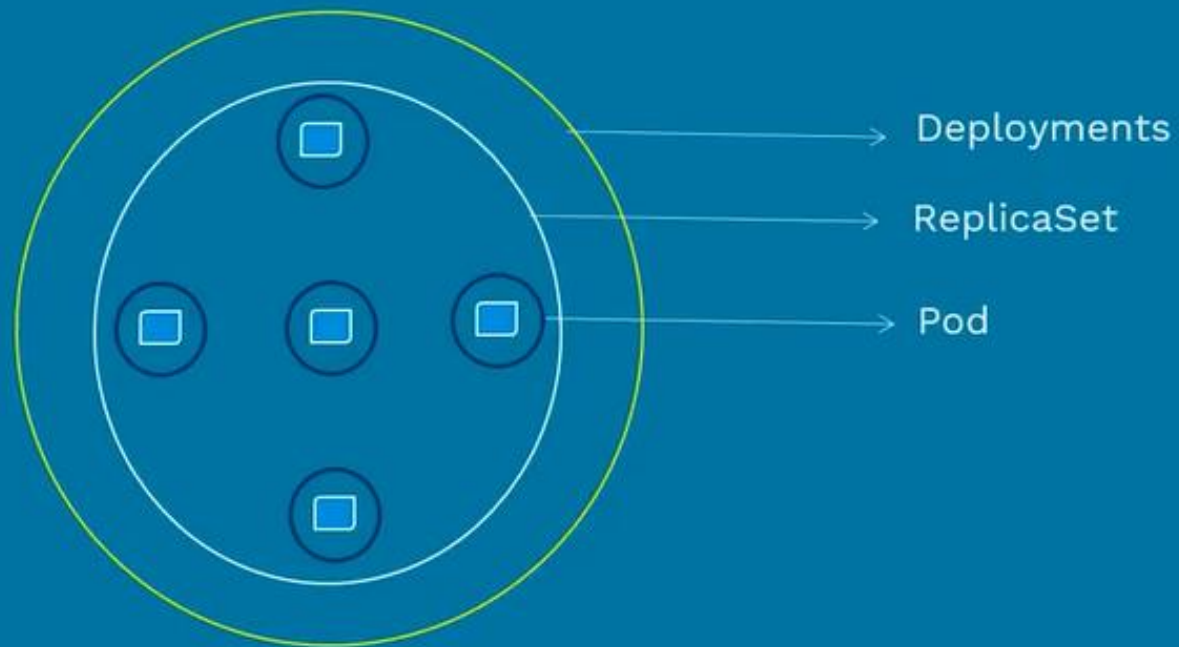
- a. Deployments Overview
- b. Features
- c. Types of Deployment

Review Demo

- a. Manifest file
- b. Deploy application with Replication Controller
- c. Display and validate
- d. Test use cases
- e. Clean up

Deployments

Updates & Rollbacks



Features

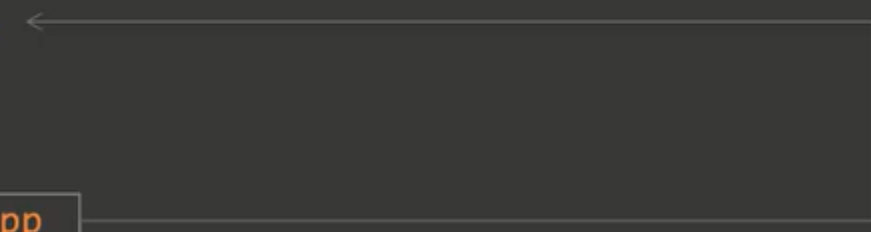
- Multiple Replicas
- Upgrade
- Rollback
- Scale up or down
- Pause and Resume

Deployment Types

- Recreate
- RollingUpdate (Ramped or Incremental)
- Canary
- Blue / Green

Deployments - Manifest file

```
# Deployment
#controllers/nginx-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
  labels:
    app: nginx-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```



The diagram illustrates the label inheritance in a Kubernetes Deployment. A long horizontal line originates from the 'app: nginx-app' label under the 'selector' section and points to the 'app: nginx-app' label under the 'template' section's 'labels' field. A shorter horizontal line originates from the 'app: nginx-app' label under the 'template' section's 'labels' field and points to the 'app: nginx-app' label under the 'spec' section's 'containers' list.

Deployments – Create & Display

```
[srinath@master ~]$ kubectl create -f nginx-deploy.yaml  
deployment.apps/nginx-deployment created
```

```
[srinath@master ~]$ kubectl get deploy -l app=nginx-app
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	3	3	3	3	1m

```
[srinath@master ~]$ kubectl get rs -l app=nginx-app
```

NAME	DESIRED	CURRENT	READY	AGE
nginx-deployment-c75f4bb64	3	3	3	3m

```
[srinath@master ~]$ kubectl get po -l app=nginx-app
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-c75f4bb64-h7hph	1/1	Running	0	8m
nginx-deployment-c75f4bb64-pbmj4	1/1	Running	0	8m
nginx-deployment-c75f4bb64-sr4vt	1/1	Running	0	8m

Deployments – Describe

```
[srinath@master ~]$ kubectl describe deploy nginx-deployment
```

```
Name:                nginx-deployment
Namespace:           default
CreationTimestamp:    Wed, 29 Aug 2018 11:39:31 +0000
Labels:              app=nginx-app
Annotations:         deployment.kubernetes.io/revision=1
Selector:            app=nginx-app
Replicas:            3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx-app
  Containers:
    nginx-container:
      Image:        nginx:1.7.9
      Port:         80/TCP
  ...
  ...
Conditions:
  Type           Status  Reason
  ----           -
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-c75f4bb64 (3/3 replicas created)
Events:
  Type           Reason             Age   From                      Message
  ----           -
  Normal         ScalingReplicaSet   13m   deployment-controller     Scaled up replica set nginx-deployment-c75f4bb64 to 3
```


Update

nginx:1.7.9 -> nginx:1.9.1

Deployments – Update Deployment

```
[srinath@master ~]$ kubectl set image deploy nginx-deployment nginx-container=nginx:1.9.1  
deployment.extensions/nginx-deployment image updated
```

```
[srinath@master ~]$ kubectl edit deploy nginx-deployment  
deployment.extensions/nginx-deployment image updated
```

```
[srinath@master ~]$ kubectl rollout status deployment/nginx-deployment  
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...  
Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...  
deployment "nginx-deployment" successfully rolled out
```

```
[srinath@master ~]$ kubectl get deploy
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	3	3	3	3	48m

Rollback

nginx:1.7.9 -> nginx:1.91

Deployments – Rollback Deployment

```
[srinath@master ~]$ kubectl set image deploy nginx-deployment nginx-container=nginx:1.91 --record  
deployment.extensions/nginx-deployment image updated
```

```
[srinath@master ~]$ kubectl rollout status deployment/nginx-deployment  
Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
```

```
[srinath@master ~]$ kubectl rollout history deployment/nginx-deployment  
deployments "nginx-deployment"  
REVISION  CHANGE-CAUSE  
1  kubectl create --filename=nginx-deploy.yaml --record=true  
2  kubectl set image deploy nginx-deployment nginx-container=nginx:1.91 --record=true
```

```
[srinath@master ~]$ kubectl rollout undo deployment/nginx-deployment  
deployment.extensions/nginx-deployment
```

```
[srinath@master ~]$ kubectl rollout status deployment/nginx-deployment  
deployment "nginx-deployment" successfully rolled out
```

Scale up & down

Deployments – Scaling up

```
[srinath@master ~]$ kubectl scale deployment nginx-deployment --replicas=5  
deployment.extensions/nginx-deployment scaled
```

```
[srinath@master ~]$ kubectl get deploy
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	5	5	5	5	20m

```
[srinath@master ~]$ kubectl get po
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-c75f4bb64-8dcmw	1/1	Running	0	3m
nginx-deployment-c75f4bb64-8dlb9	1/1	Running	0	21m
nginx-deployment-c75f4bb64-fxlrw	1/1	Running	0	3m
nginx-deployment-c75f4bb64-kxvtx	1/1	Running	0	3m
nginx-deployment-c75f4bb64-r6pjl	1/1	Running	0	3m

Deployments – Scaling down

```
[srinath@master ~]$ kubectl scale deployment nginx-deployment --replicas=1  
deployment.extensions/nginx-deployment scaled
```

```
[srinath@master ~]$ kubectl get deploy
```

NAME	DESIRED	CURRENT	UP-TO-DATE	AVAILABLE	AGE
nginx-deployment	1	1	1	1	24m

```
[srinath@master ~]$ kubectl get po -l app=nginx-app
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-c75f4bb64-h7hph	1/1	Running	0	8m

Deployments – Delete

```
[srinath@master ~]$ kubectl delete -f nginx-deploy.yaml  
deployment.apps "nginx-deployment" deleted
```

```
[srinath@master ~]$ kubectl get po -l app=nginx-app  
No resources found
```

Imagine that, you have been asked to deploy web app

How does this front end web app is exposed to outside world?

How do front end web app is connected to backend database?

How do we resolve Pod IP changes, when they die?

Objectives

Concept

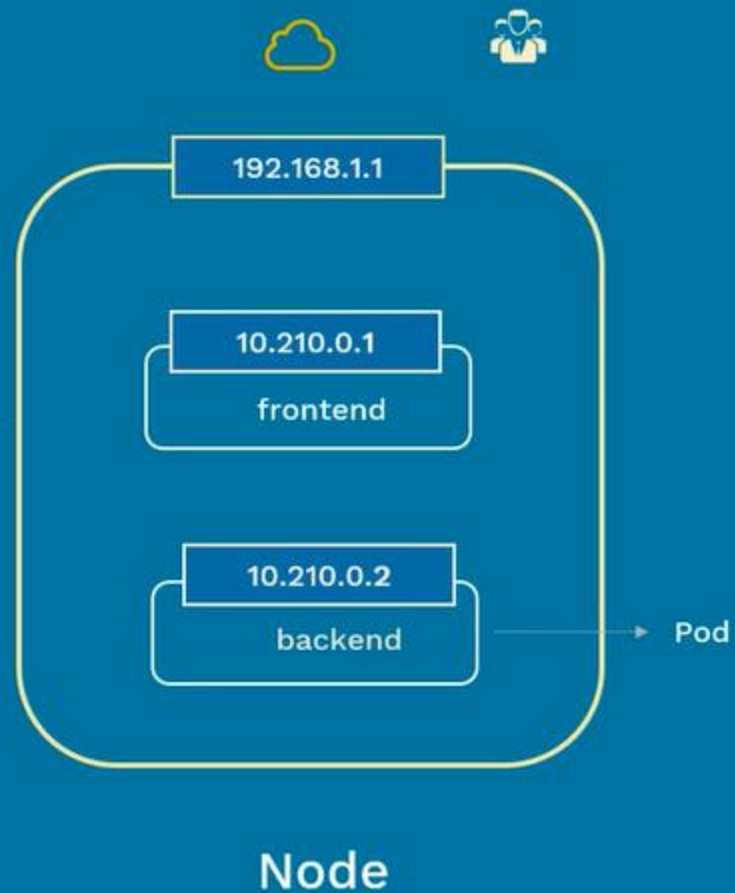
- a. Why we need Services
- b. What is a Service
- c. Types of Services

Think time...

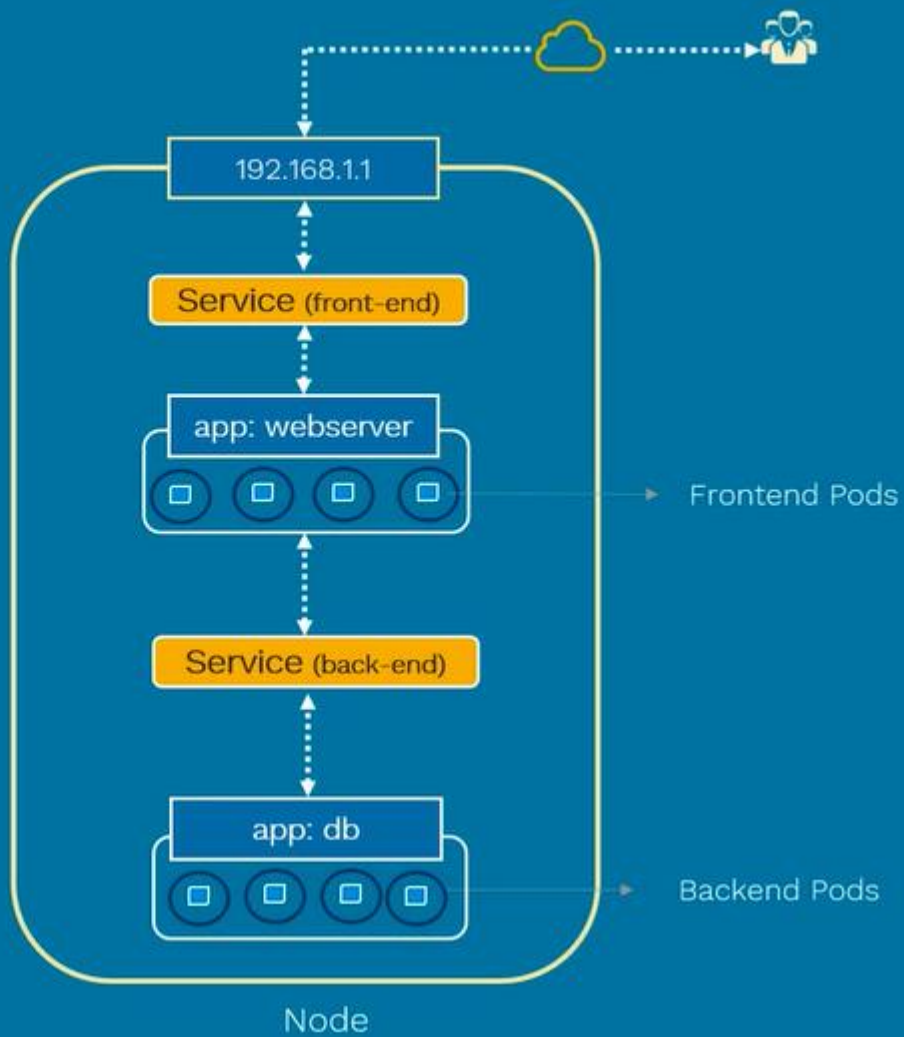
Is it possible to have permanent IP address ?

How do various components connect & communicate ?

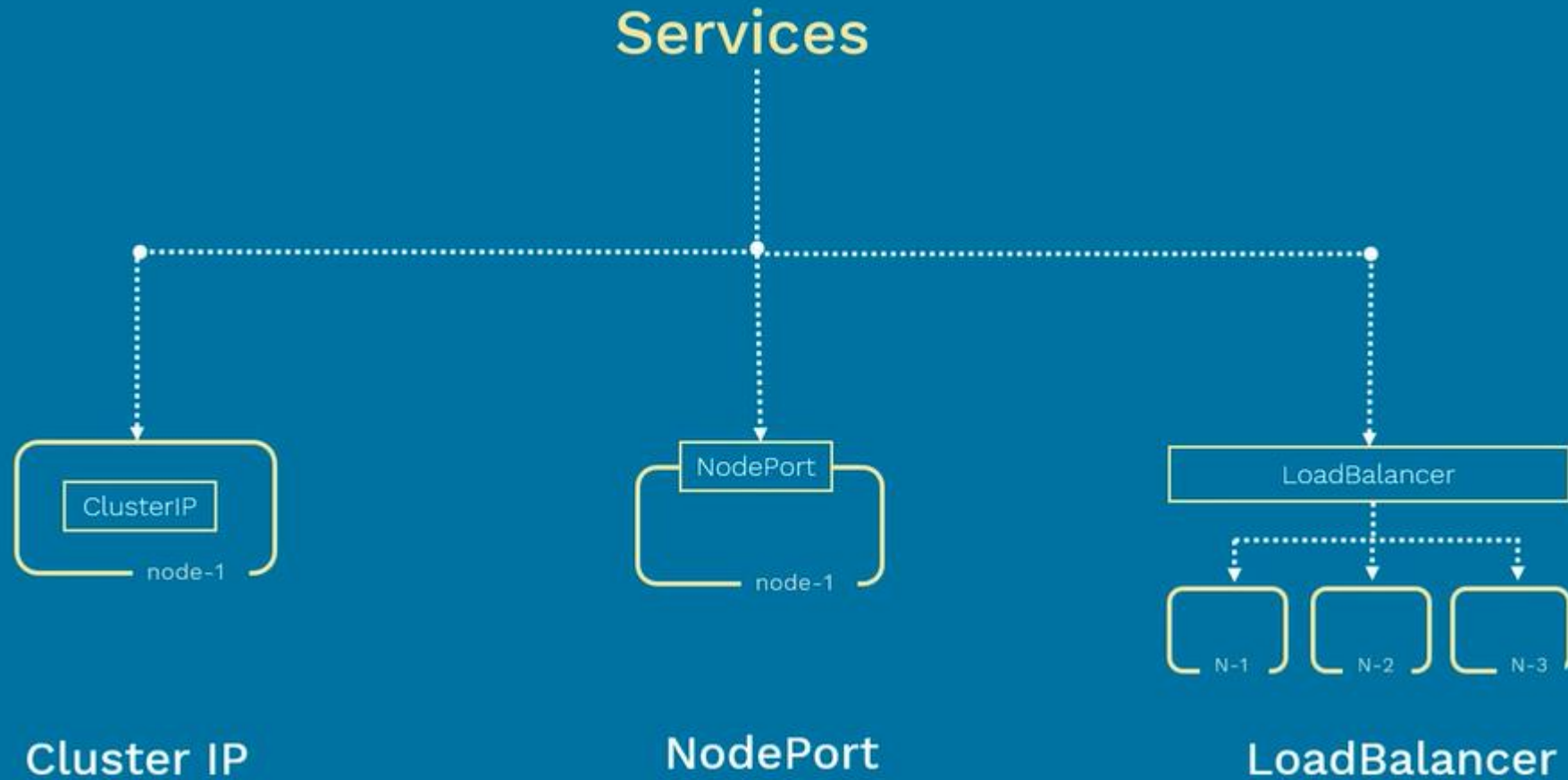
How do applications are exposed to outside world?



Services



Types of Services

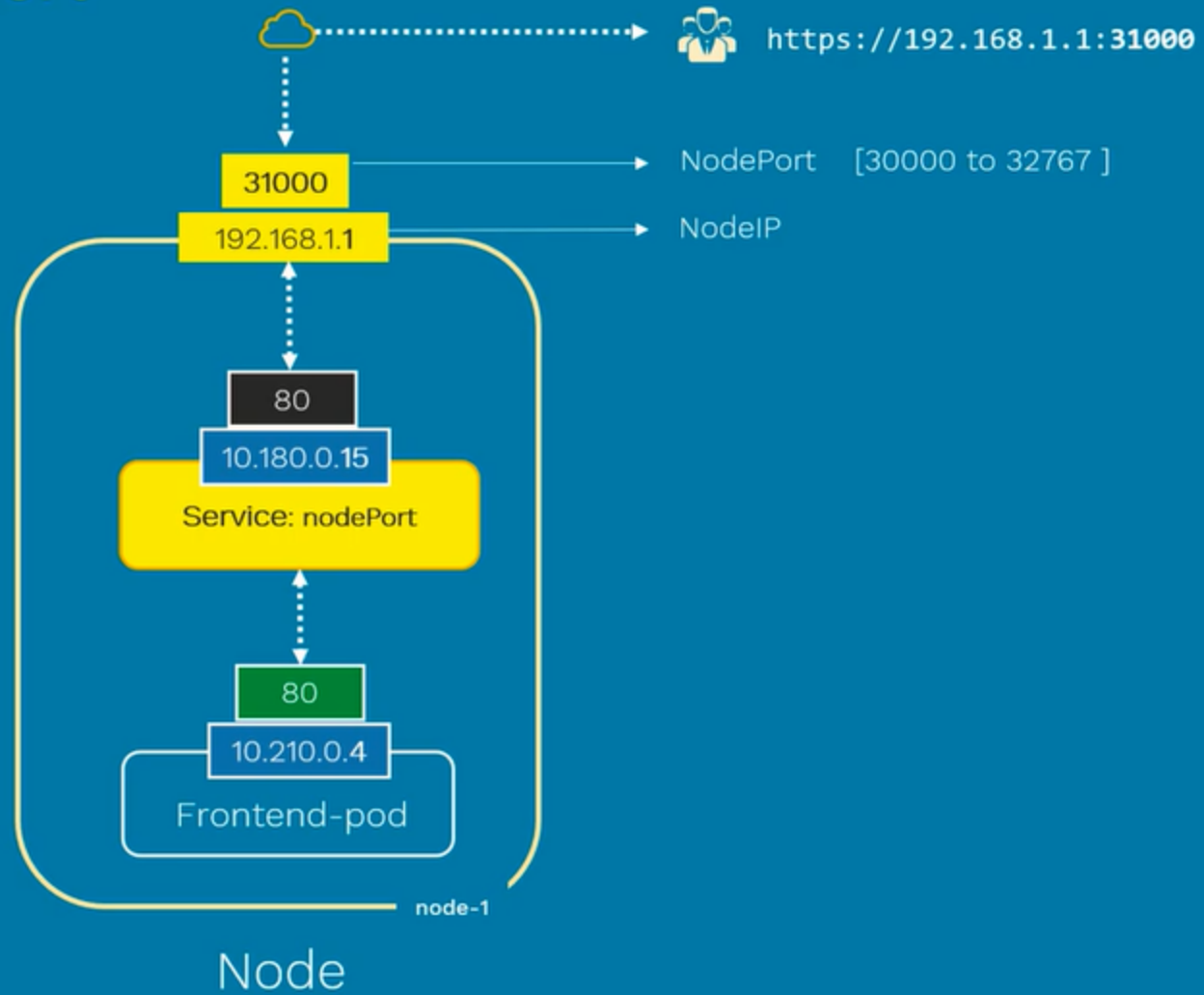


Objectives

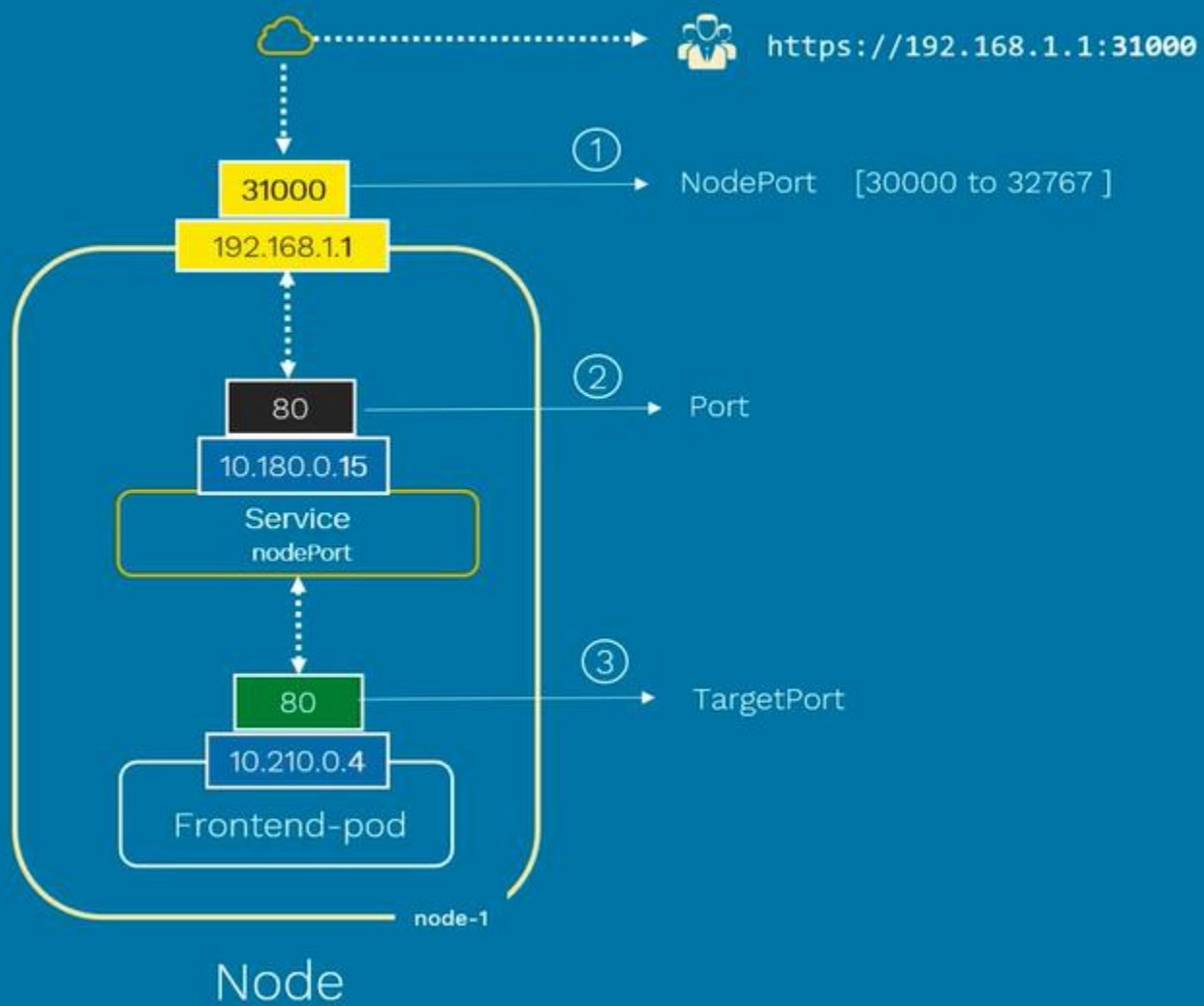
Concept

- a. Why we need?
- b. nodePort
- c. Port Types
- d. Scenarios

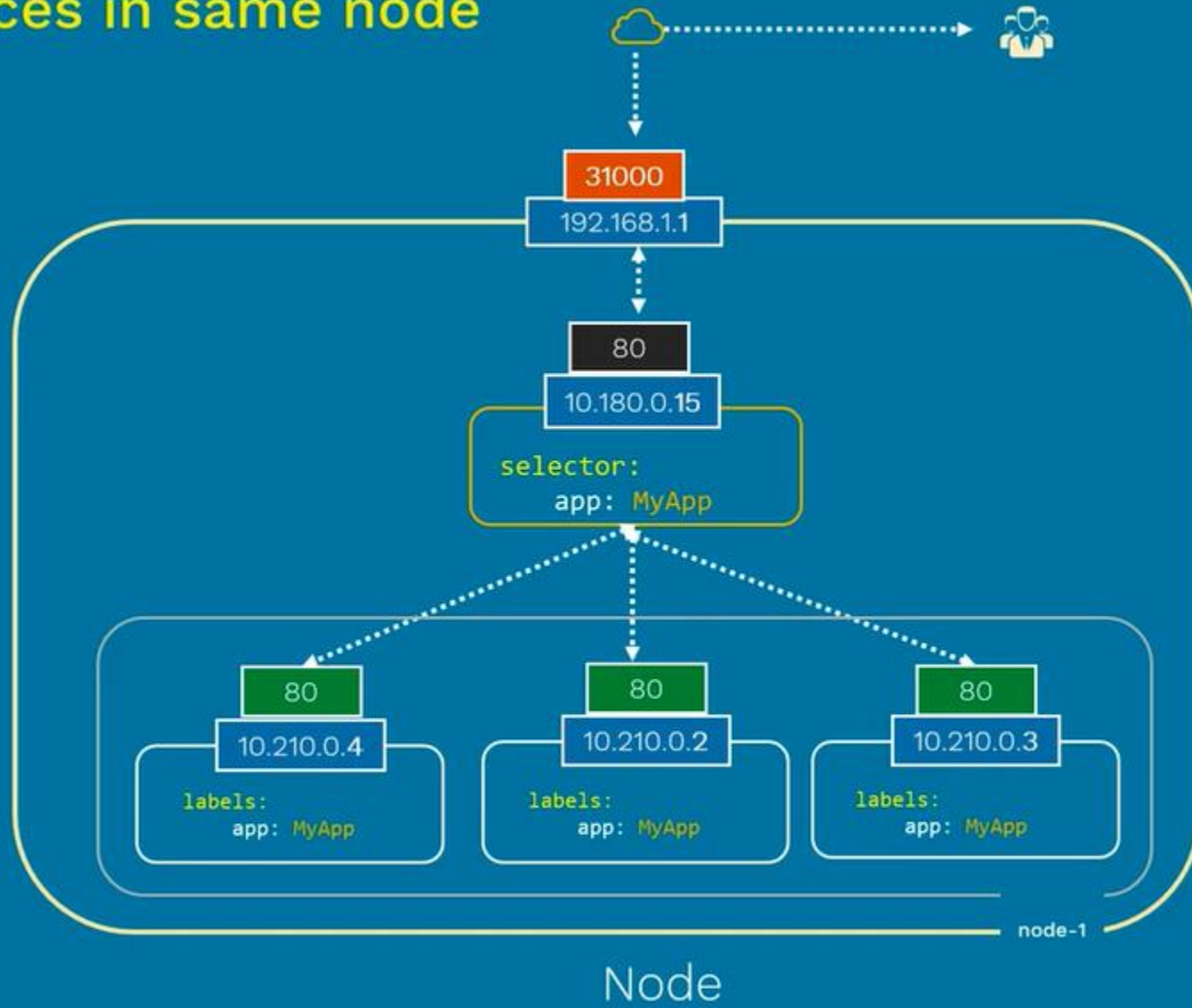
Service Type: NodePort



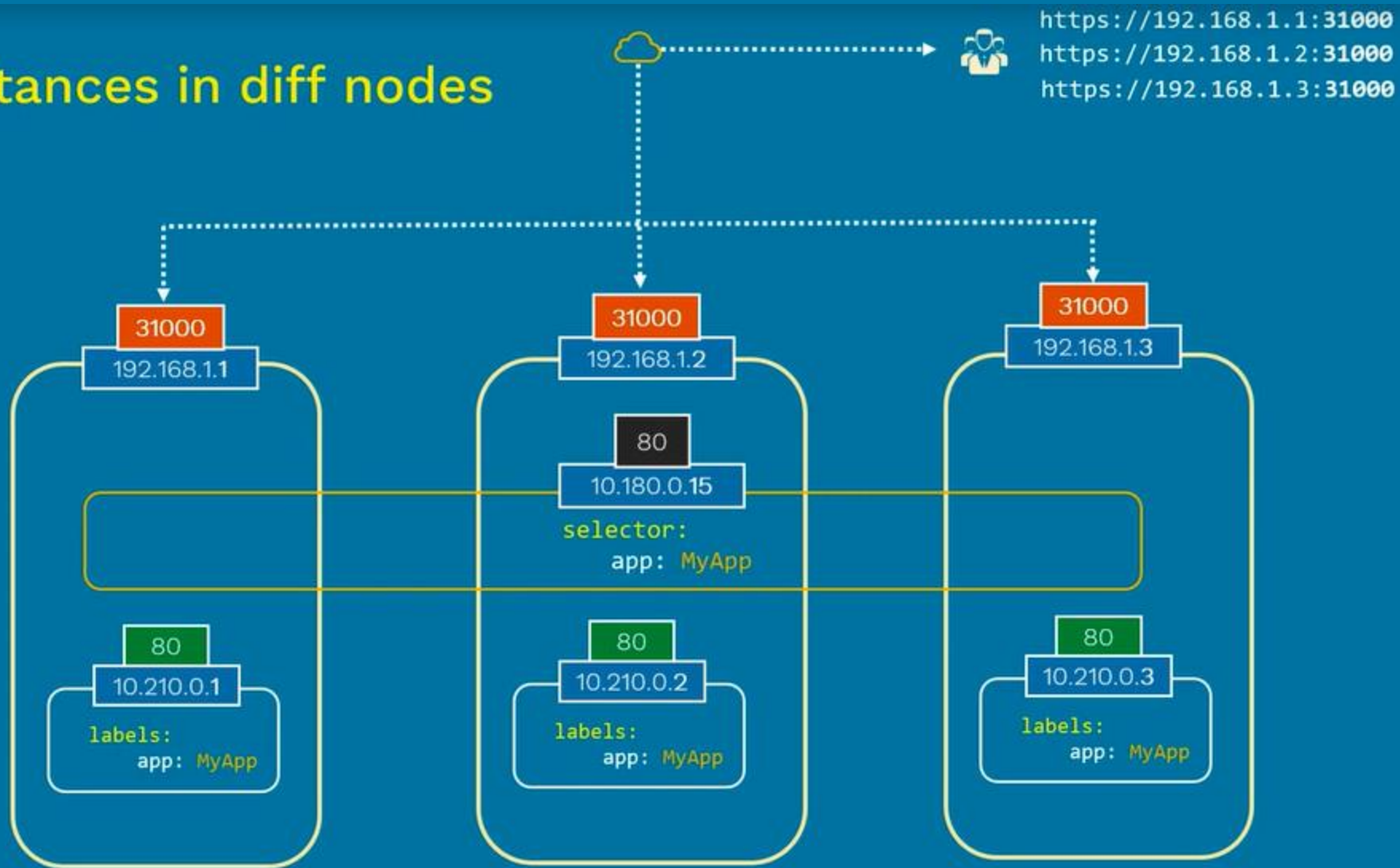
Port Types



Multi Instances in same node



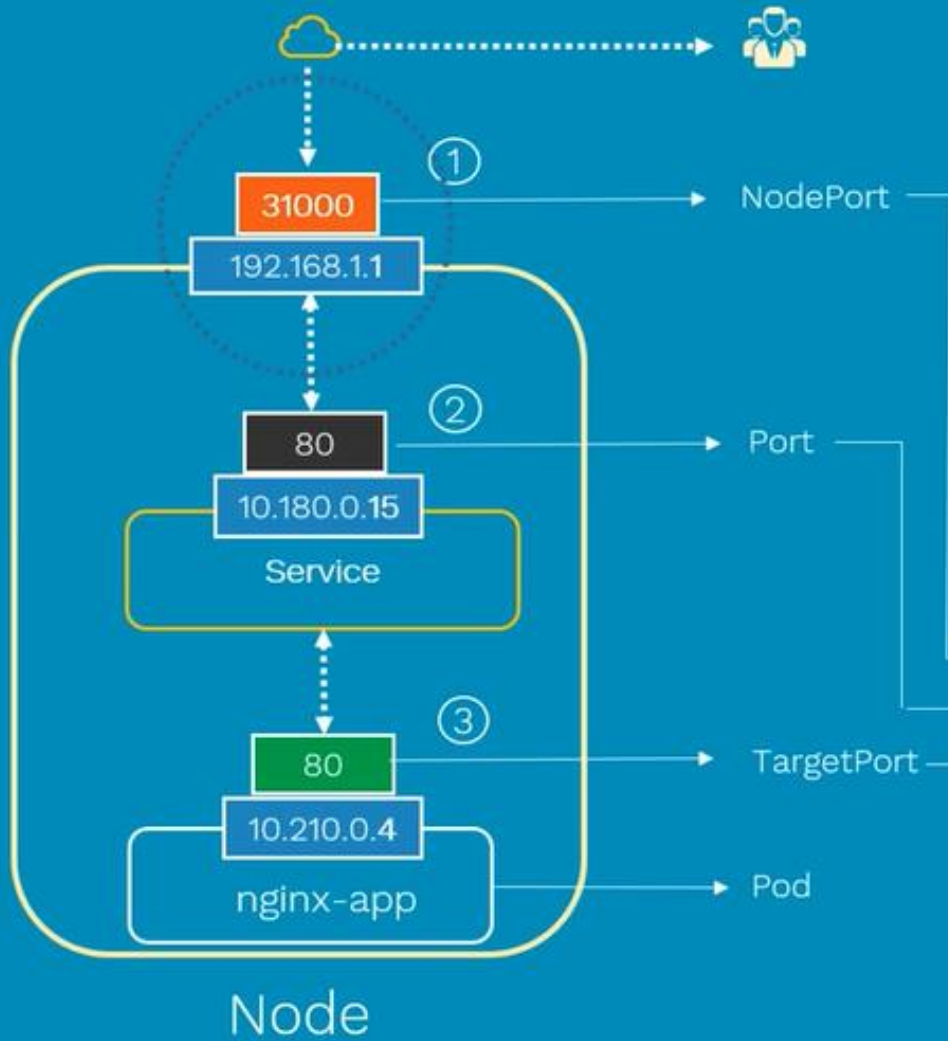
Multi Instances in diff nodes



Downsides

- You can only have once service per port
- You can only use ports 30000–32767
- If your Node/VM IP address change, you need to deal with that

Service Type: NodePort



```
# Service
# nginx-svc-np.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
  labels:
    app: nginx-app
spec:
  selector:
    app: nginx-app
  type: NodePort
  ports:
  - nodePort: 31000
    port: 80
    targetPort: 80
```

```
# Deployment
# controllers/nginx-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

NodePort – Create & Display

```
[srinath@master ~]$ kubectl create -f nginx-deploy.yaml  
deployment.apps/nginx-deployment created
```

```
[srinath@master ~]$ kubectl create -f nginx-svc-np.yaml  
service/my-service created
```

```
[srinath@master ~]$ kubectl get service -l app=nginx-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	NodePort	10.97.148.44	<none>	80:31000/TCP	14s

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
nginx-deployment-c75f4bb64-cgg7g	1/1	Running	0	12m	10.240.2.148	node2	<none>

NodePort – Describe

```
[srinath@master ~]$ kubectl describe svc my-service
```

```
Name: my-service
Namespace: default
Labels: app=nginx-app
Annotations: <none>
Selector: app=nginx-app
Type: NodePort
IP: 10.97.148.44
Port: <unset> 80/TCP
TargetPort: 80/TCP
NodePort: <unset> 31000/TCP
Endpoints: 10.240.2.148:80
Session Affinity: None
External Traffic Policy: Cluster
Events: <none>
```

NodePort – Accessing using Pod IP

```
[srinath@master ~]$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE
nginx-deployment-c75f4bb64-cgg7g	1/1	Running	0	12m	10.240.2.148	node2	<none>

```
[srinath@master ~]$ curl http://10.240.2.148:80
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Welcome to nginx!</title>
```

```
<style>
```

```
  body {
```

```
    width: 35em;
```

```
    margin: 0 auto;
```

```
    font-family: Tahoma, Verdana, Arial, sans-serif;
```

```
  }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to nginx!</h1>
```


NodePort – Accessing using Service IP

```
[srinath@master ~]$ kubectl get svc -l app=nginx-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	NodePort	10.97.148.44	<none>	80:31000/TCP	14s

```
[srinath@master ~]$ curl http://10.97.148.44:80
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Welcome to nginx!</title>
```

```
<style>
```

```
  body {
```

```
    width: 35em;
```

```
    margin: 0 auto;
```

```
    font-family: Tahoma, Verdana, Arial, sans-serif;
```

```
  }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to nginx!</h1>
```

Imagine that, using nodePort service type
you exposed your web app to outside world on the internet

which node IP and nodePort will you provide to end users?

Objectives

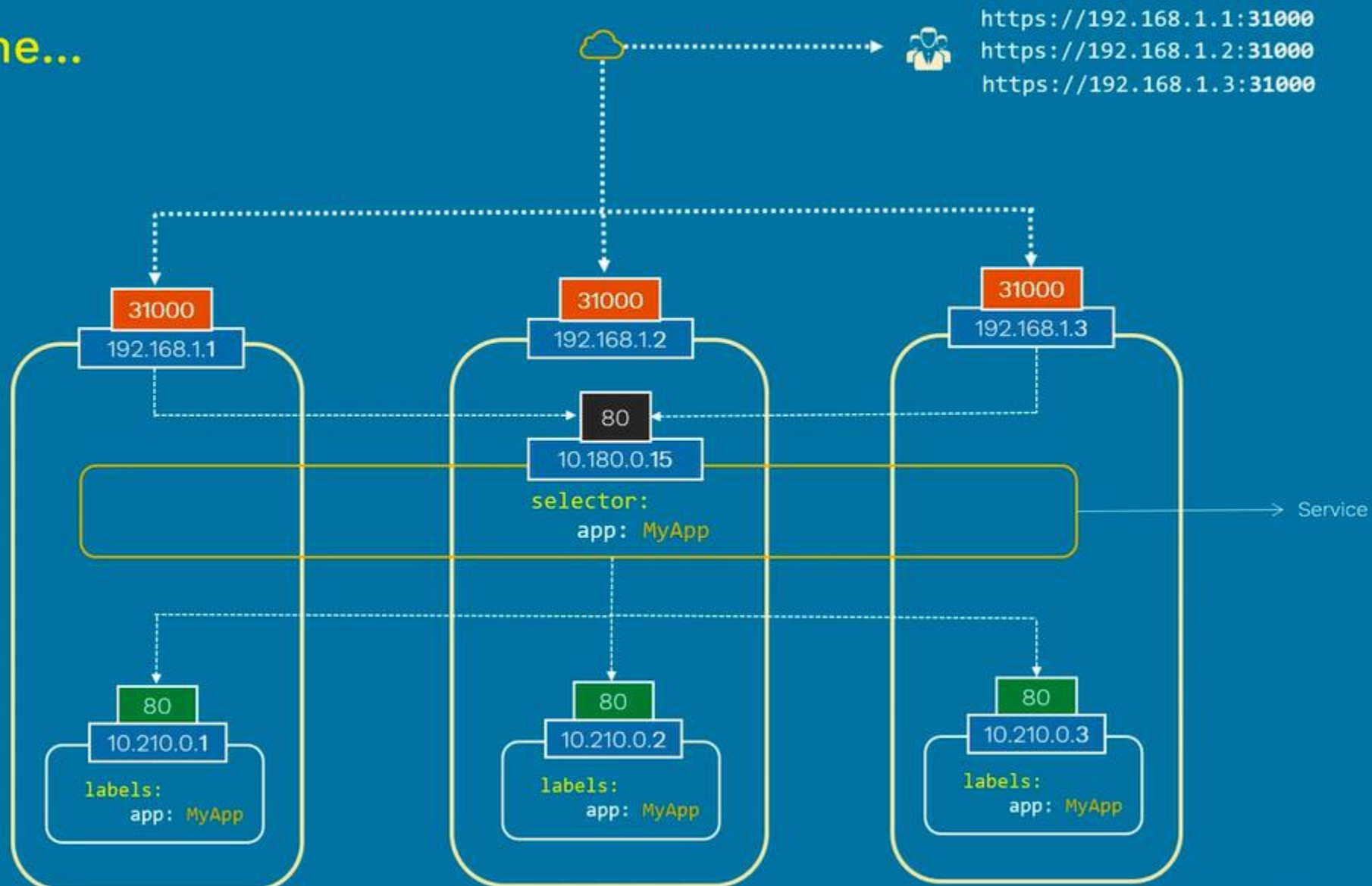
Concept

- a. Why we need?
- b. LoadBalancer - Overview

Review Demo

- a. Manifest file
- b. Create and display
- c. Test usecase
- d. Clean up

Think time...



LoadBalancer – Config

```
# Service - LoadBalancer
# nginx-service -lb.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
  labels:
    app: nginx-app
spec:
  selector:
    app: nginx-app
  type: LoadBalancer
  ports:
  - nodePort: 31000
    port: 80
    targetPort: 80
```

```
# Deployment
# controllers/nginx-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-app
  template:
    metadata:
      labels:
        app: nginx-app
    spec:
      containers:
      - name: nginx-container
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```



LoadBalancer – Create & Display

```
srinath@master:~ $ kubectl create -f nginx-deploy.yaml  
deployment.apps/nginx-deployment created
```

```
srinath@master:~ $ kubectl create -f nginx-service -lb.yaml  
service/my-service created
```

```
srinath@master:~ $ kubectl get service -l app=nginx-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	LoadBalancer	10.11.241.216	<pending>	80:31000/TCP	26s

LoadBalancer – LoadBalancer - Display

```
srinath@master:~ $ kubectl describe service my-service
```

```
Name: my-service
Namespace: default
Labels: app=nginx-app
Annotations: <none>
Selector: app=nginx-app
Type: LoadBalancer
IP: 10.11.241.216
LoadBalancer Ingress: 104.198.43.137
Port: <unset> 80/TCP
TargetPort: 80/TCP
NodePort: <unset> 31000/TCP
Endpoints: 10.8.0.12:80
Session Affinity: None
External Traffic Policy: Cluster
```

```
Events:
```

Type	Reason	Age	From	Message
----	-----	----	----	-----
Normal	EnsuringLoadBalancer	15m	service-controller	Ensuring load balancer
Normal	EnsuredLoadBalancer	14m	service-controller	Ensured load balancer

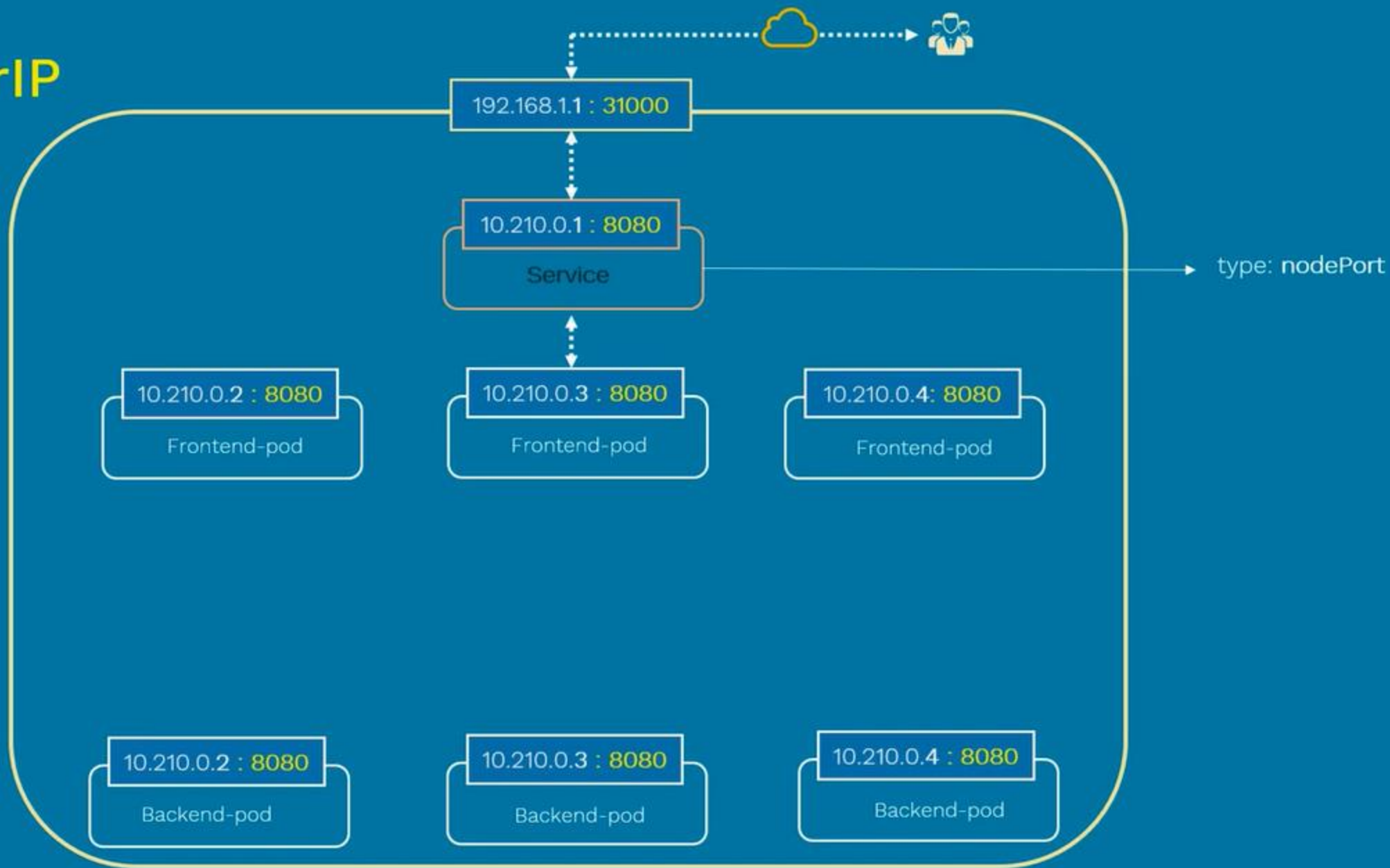
LoadBalancer – Deleting Service

```
[srinath@master ~]$ kubectl delete service my-service  
service "my-service" deleted
```

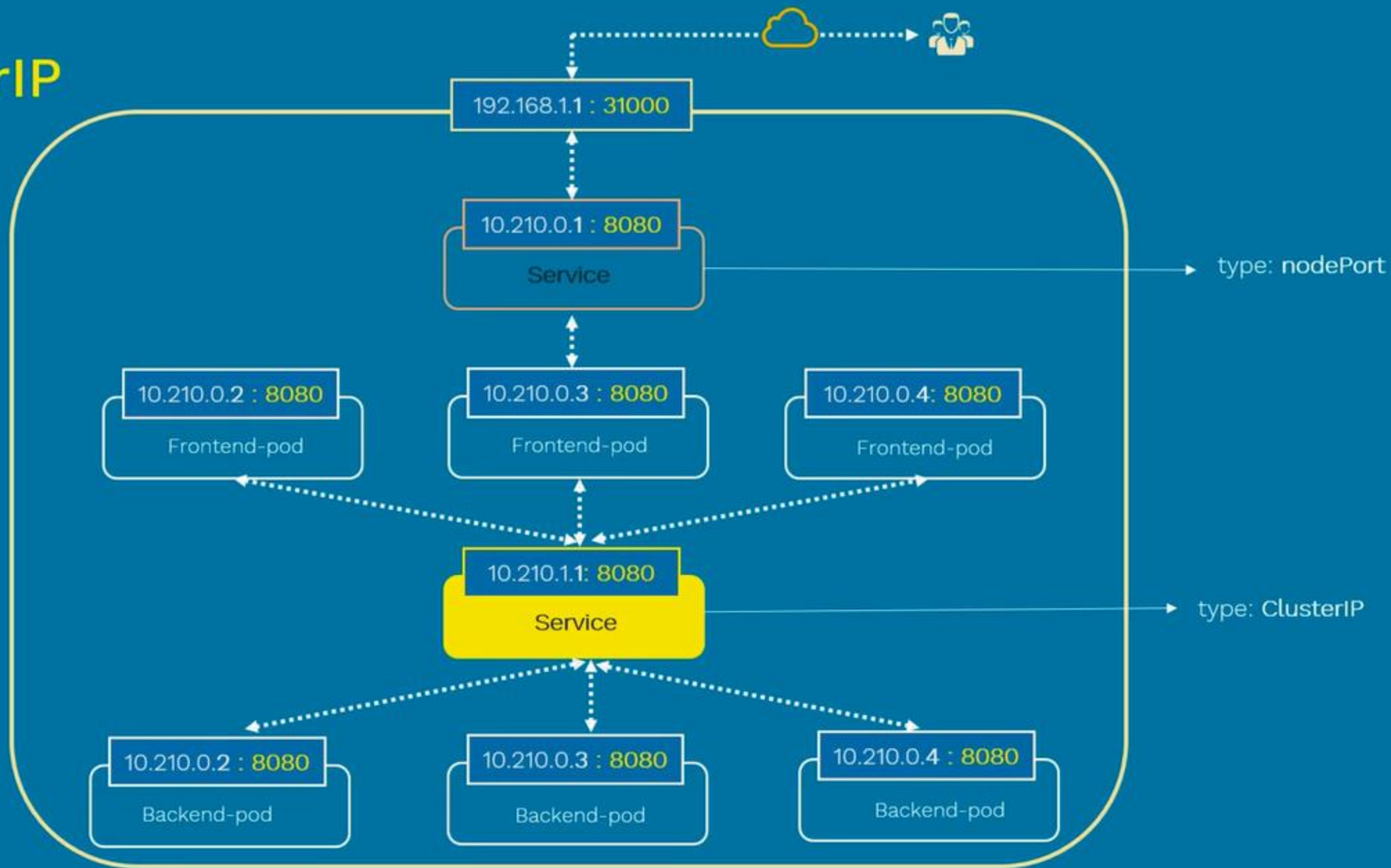
```
[srinath@master ~]$ kubectl get pods  
No resources found
```

Imagine that, you need to deploy one full fledged app which consists of frontend application and backend database

ClusterIP



ClusterIP



① Deploying 1 Redis DB Master

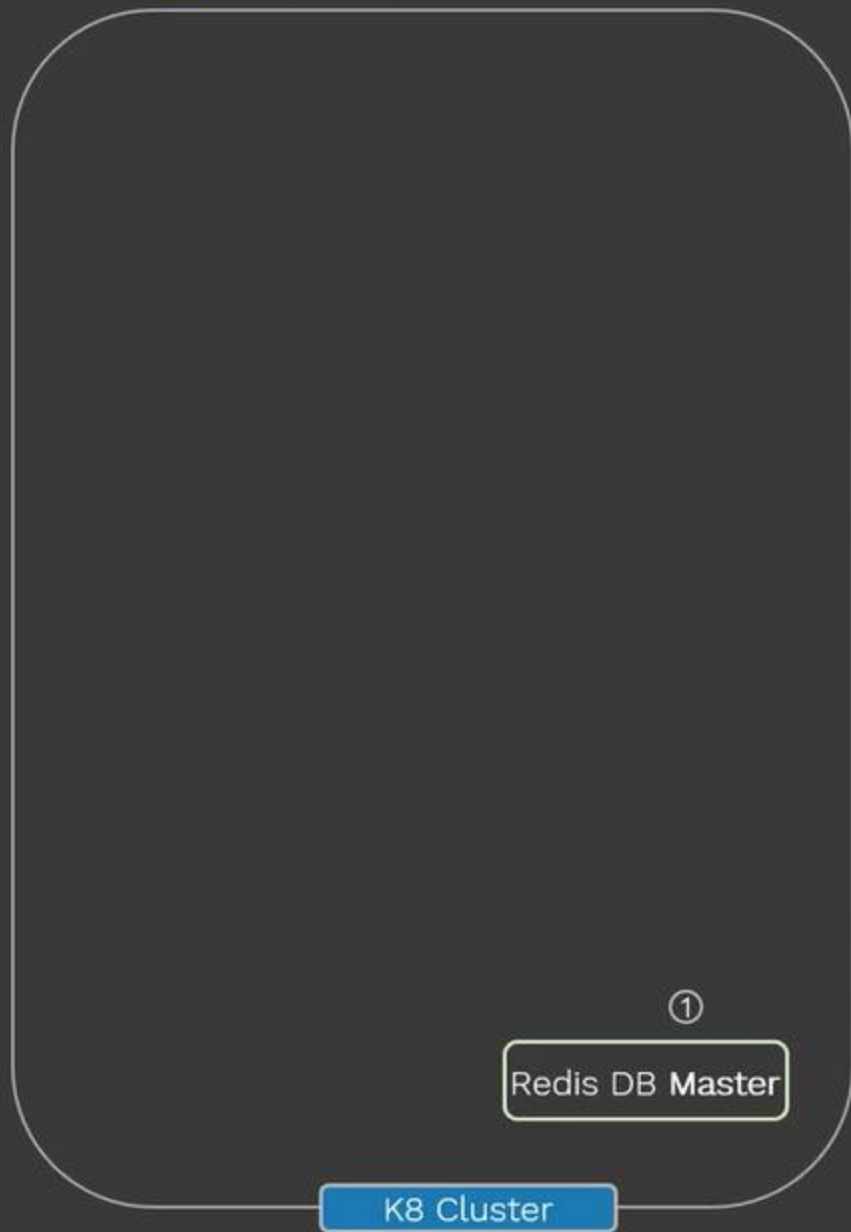


```
# redis-master-deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-master
  labels:
    app: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
      role: master
      tier: backend
  template:
    metadata:
      labels:
        app: redis
        role: master
        tier: backend
    spec:
      containers:
      - name: master
        image: k8s.gcr.io/redis:e2e
        resources:
          requests:
            cpu: 100m
            memory: 100Mi
        ports:
        - containerPort: 6379
```

ReplicaSet

Pod

② Deploying 3 Redis DB Slaves

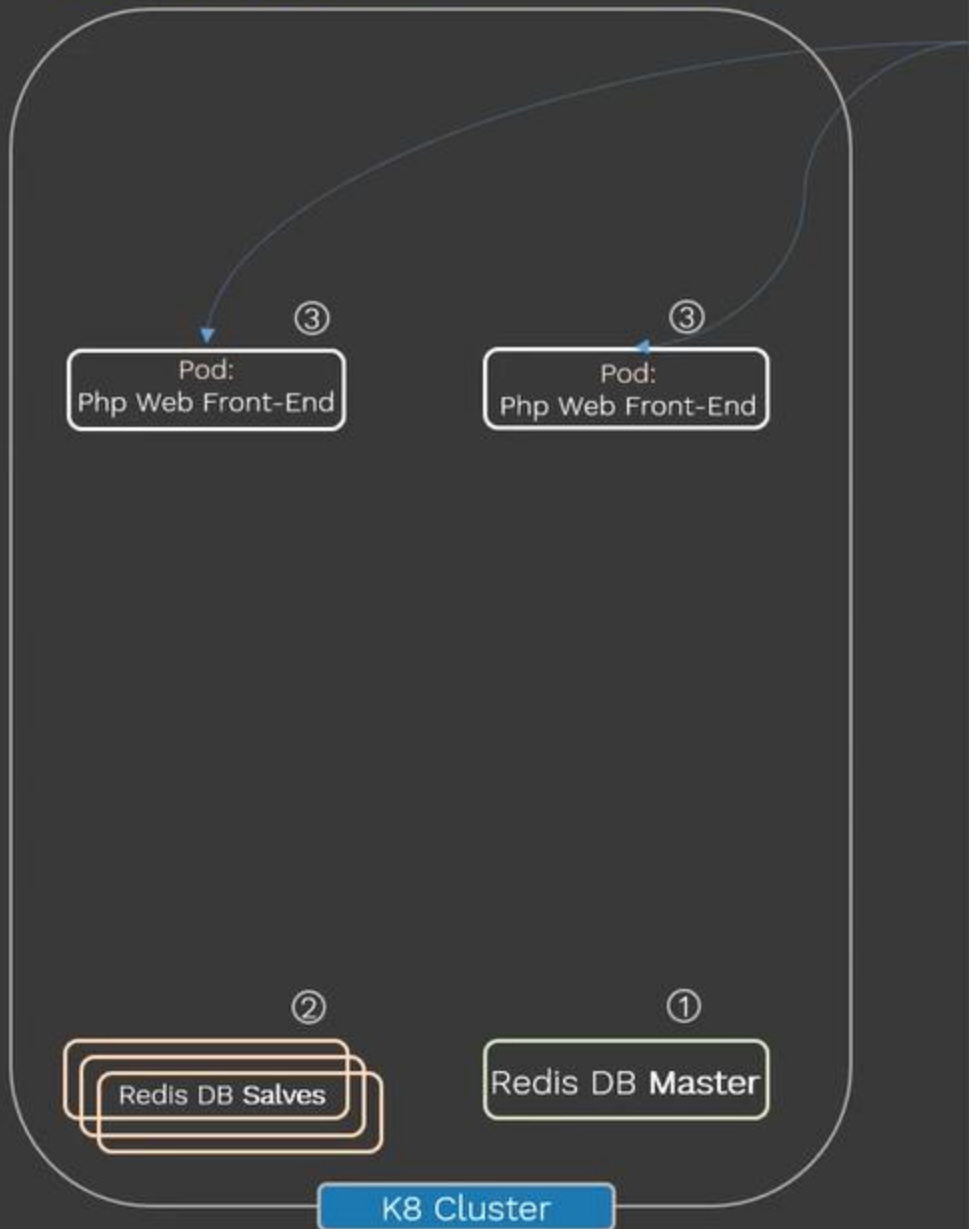


```
# redis-slave.yaml
apiVersion: apps/v1 # for versions before 1.9.0 use apps/v1beta2
kind: Deployment
metadata:
  name: redis-slave
  labels:
    app: redis
spec:
  replicas: 3
  selector:
    matchLabels:
      app: redis
      role: slave
      tier: backend
  template:
    metadata:
      labels:
        app: redis
        role: slave
        tier: backend
    spec:
      containers:
        - name: slave
          image: gcr.io/google_samples/gb-redisslave:v1
          resources:
            requests:
              cpu: 100m
              memory: 100Mi
          ports:
            - containerPort: 6379
```

ReplicaSet

Pod

③ Deploying 2 front-end PHP web pods

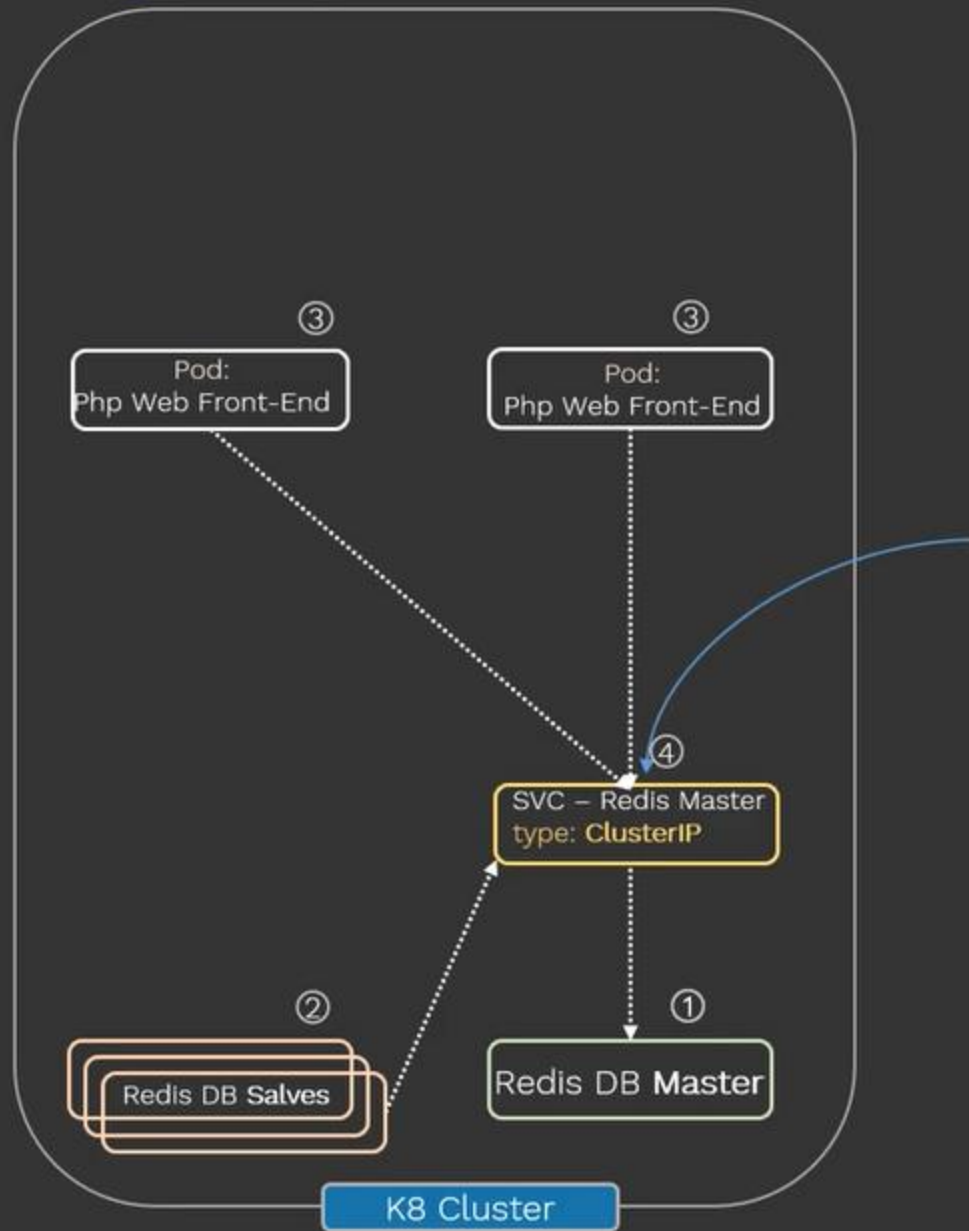


```
# redis-master-deployment.yaml
apiVersion: apps/v1 # for versions before 1.9.0 use apps/
kind: Deployment
metadata:
  name: frontend
  labels:
    app: guestbook
spec:
  replicas: 2
  selector:
    matchLabels:
      app: guestbook
      tier: frontend
  template:
    metadata:
      labels:
        app: guestbook
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: gcr.io/google-samples/gb-frontend:v4
          resources:
            requests:
              cpu: 100m
              memory: 100Mi
          ports:
            - containerPort: 80
```

ReplicaSet

Pod

④ Creating Service for Redis Master



```
# redis-master-service.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: redis-master-svc
```

```
  labels:
```

```
    app: redis
```

```
    role: master
```

```
    tier: backend
```

```
spec:
```

```
  ports:
```

```
    - port: 6379
```

```
      targetPort: 6379
```

```
  type: ClusterIP
```

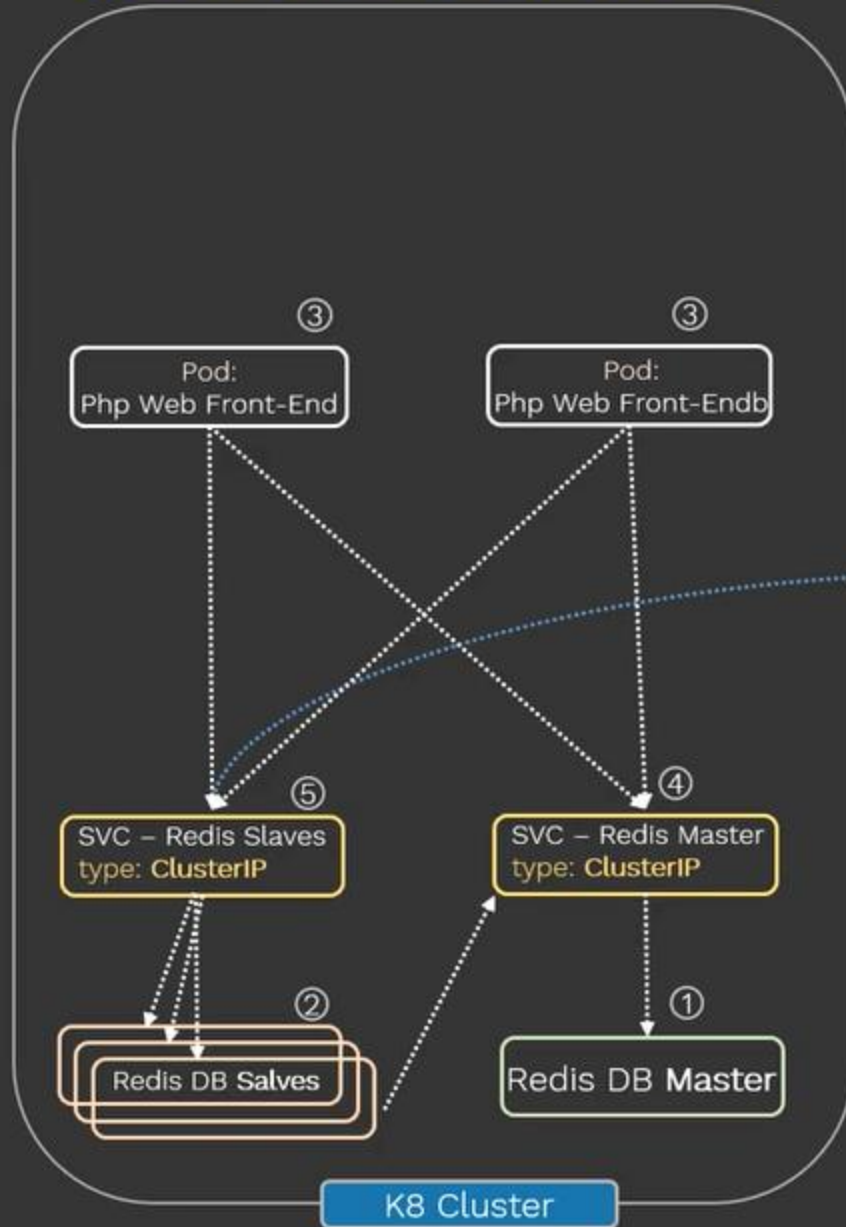
```
  selector:
```

```
    app: redis
```

```
    role: master
```

```
    tier: backend
```

④ Creating Service for Redis DB Slaves



```
# redis-master-deployment.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: redis-slave-svc
```

```
  labels:
```

```
    app: redis
```

```
    role: slave
```

```
    tier: backend
```

```
spec:
```

```
  ports:
```

```
  - port: 6379
```

```
  type: ClusterIP
```

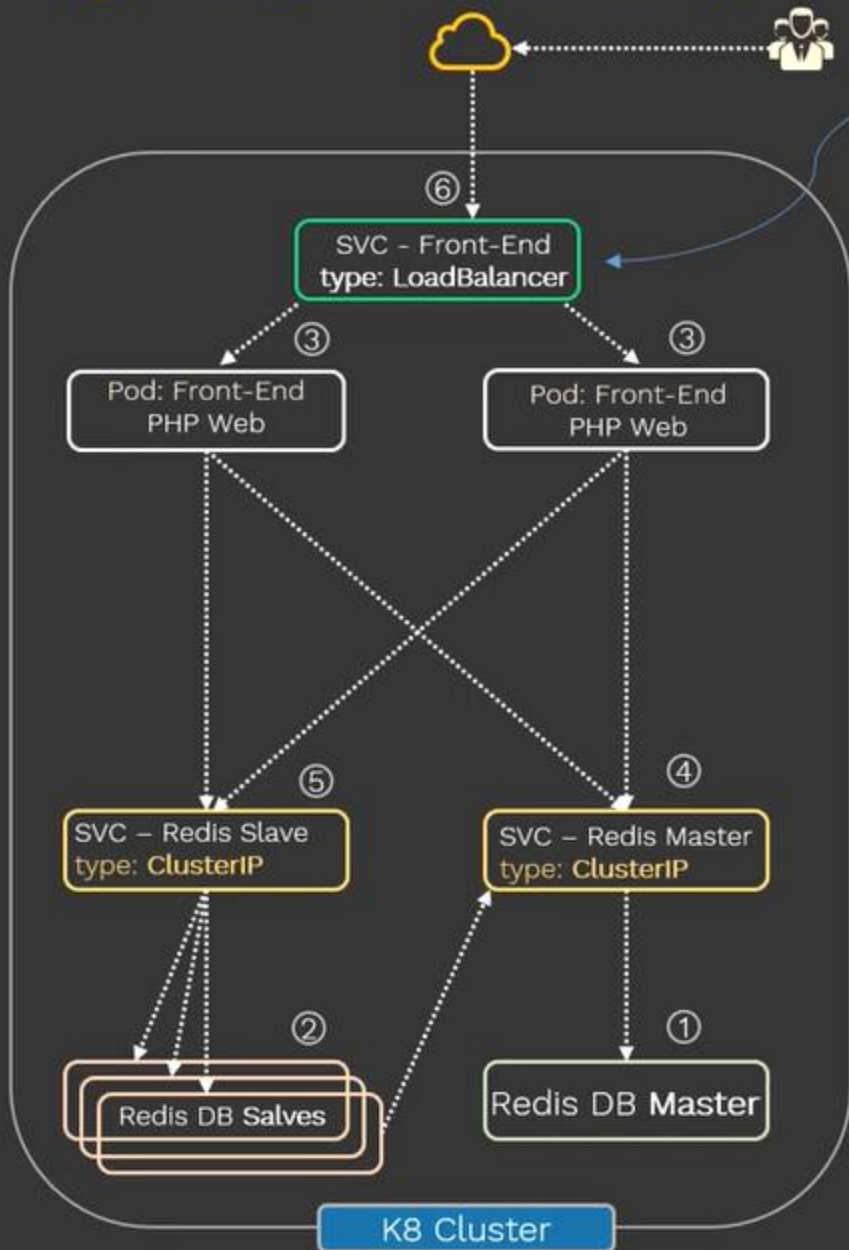
```
  selector:
```

```
    app: redis
```

```
    role: slave
```

```
    tier: backend
```

⑥ Creating Service for front-end apps



```
# redis-master-deployment.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
  labels:
    app: guestbook
    tier: frontend
spec:
  type: LoadBalancer
  ports:
    - port: 80
  selector:
    app: guestbook
    tier: frontend
```

NOTE: if your cluster doesn't support LB, use type as NP #
type: NodePort

Services –Display

```
srinath@master:~ $ kubectl get po -l tier=backend
```

NAME	READY	STATUS	RESTARTS	AGE
redis-master-585798d8ff-bx5z6	1/1	Running	0	1h
redis-slave-5dfddd78f5-4f7gc	1/1	Running	0	1h
redis-slave-5dfddd78f5-7z84b	1/1	Running	0	1h
redis-slave-5dfddd78f5-bwg74	1/1	Running	0	1h

```
srinath@master:~ $ kubectl get po -l tier=frontend
```

NAME	READY	STATUS	RESTARTS	AGE
frontend-d5d5947-7skmx	1/1	Running	0	1h
frontend-d5d5947-vfc9w	1/1	Running	0	1h

```
srinath@master:~ $ kubectl get svc -l tier=backend
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
redis-master-svc	ClusterIP	10.11.240.185	<none>	6379/TCP	11m
redis-slave-svc	ClusterIP	10.11.252.193	<none>	6379/TCP	10m

```
srinath@master:~ $ kubectl get svc -l tier=frontend
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
frontend	LoadBalancer	10.11.250.88	104.198.241.135	80:31612/TCP	8m

